AMAZE FILE MANAGER SECURITY ANALYSIS: A CONTROLLED SIMULATION OF TROJAN INJECTION AND REVERSE ENGINEERING

D.M.M.P.S. Mahamohottala
Bachelor of Science (Hons) in Information Technology Specialized in Cyber Security

Udith Kaushalya
Bachelor of Science (Hons) in Information Technology Specialized in Cyber Security

Jayasinghe J.M.S.S
Bachelor of Science (Hons) in Information Technology Specialized in Cyber Security

Peiris W.S.S.N
Bachelor of Science (Hons) in Information Technology Specialized in Cyber Security

Department of Information Technology
Sri Lanka Institute of Information Technology
Sri Lanka

April 2026

ABSTRACT

This study demonstrates the reverse engineering and Trojan injection of an Android application to analyze potential security risks associated with application tampering. The selected application, Amaze File Manager, was decompiled to examine its internal structure, including activities, services, permissions, and execution flow. The reverse engineering process enabled identification of suitable injection points for integrating malicious functionality without disrupting the original behavior of the application.

A malicious background service named DemoService was designed and injected into the application. The service simulates Trojan-like behavior by collecting device information, executing a CPU-intensive workload, and performing unauthorized background operations. The malicious component was registered in the AndroidManifest.xml file and triggered through modifications to the MainActivity user interface. Two trigger mechanisms were implemented: a manual trigger via menu options and a time-based conditional trigger that executes the payload only during even minutes.

The modified application was rebuilt and installed in a controlled Android emulator environment. Testing was conducted to verify execution of the malicious service. Logcat output confirmed device information logging, service startup, and CPU workload execution. The application continued to operate normally while the simulated payload executed in a controlled background service.

The results highlight the risks associated with reverse engineering and malware injections, including unauthorized background execution, resource exhaustion, and hidden malicious functionality. The study emphasizes the importance of secure coding practices, application integrity verification, and code obfuscation to prevent tampering. The project concludes that Android applications without adequate protection mechanisms are vulnerable to Trojan injections and recommends implementing security controls to mitigate such threats.

Table of Contents

ABSTRACT ii
LIST OF FIGURES v
LIST OF TABLES vi
LIST OF ABBREVIATIONS viii
LIST OF APPENDICES ix
INTRODUCTION x
1.1 Background of Android Application Security x
1.2 Reverse Engineering Overview xi
1.3 Malware Injection Concept xii
1.4 Selected Application – Amaze File Manager xiii
1.5 Problem Statement xiv
1.6 Scope and Limitations xv
2. Objectives xvi
3. Methodology xvii
3.1 Overall Project Methodology xvii
3.2 Reverse Engineering Method xvii
3.3 Malware Injection Method xviii
3.4 Testing Method xix
3.5 Tools and Technologies Used xix
4. Development and Testing Environment xxi
4.1 Hardware Environment xxi
4.2 Software Environment xxi
4.3 Emulator Setup xxii
5. Reverse Engineering Analysis xxiv
5.1 APK Structure Analysis xxiv
5.2 Component Mapping xxv
5.2.1 Activities Mapping xxv
5.2.2 Service Mapping xxvi
5.2.3 Broadcast Receiver Mapping xxvii
5.2.4 Permission Mapping xxviii
5.2.5 Providers xxix
5.2.6 Component Relationship xxx
5.3 Control Flow Explanation xxx
5.3.1 Application Entry Flow xxxi
5.3.2 Activity Flow xxxi
5.3.3 Service Execution Flow xxxi

5.4 Identified Vulnerabilities xxxi
5.4.1 Exported Activity Vulnerabilities xxxii
5.4.2 Background Service Execution Vulnerability xxxii
5.4.3 Excessive Permission Usage xxxii
6. Malware Injection Design xxxiii
6.1 Step by Step Injection Process xxxiii
6.1.1 Decompile APK xxxiii
6.1.2 Add Trojan Service xxxiv
6.1.3 Modify AndroidManifest.xml xxxv
6.1.4 Modify MainActivity (Trigger) xxxv
6.1.5 Rebuild APK xxxvi
6.1.6 Install Modified APK xxxvi
6.2 Code Modification with Explanation xxxvii
6.2.1 DemoService.kt (Trojan Service) xxxvii
6.2.2 MainActivity.java Modification xl
6.2.3 AndroidManifest.xml Modification xlii
6.3 Trigger Logic Implementation xliii
6.3.1 Notification Trigger (User Interaction) xliii
6.3.2 CPU Usage Spike Trigger xliv
6.3.3 Payload Selection xliv
6.4 Permission Modification xlv
6.4.1 Foreground Service Permission xlv
6.4.2 Foreground Data Sync Permission xlvi
6.4.3 Notification Permission xlvi
6.4.4 Wake Lock Permission xlvii
6.5 Service Lifecycle Explanation xlvii
6.5.1 Service Start (onStartCommand) xlviii
6.5.2 Mode Detection xlviii
6.5.3 Foreground Service Execution xlviii
6.5.4 Background Thread Execution xlviii
6.5.5 CPU Workload Execution xlix
6.5.6 Service Stop xlix
7. Security Analysis xlix
7.1 How Could This Attack Be Prevented? xlix
7.1.1 Code Obfuscation xlix
7.1.2 APK Integrity Checking l
7.1.3 Disable Exported Components l

7.1.4 Restrict Background Services l
7.1.5 Anti Reverse Engineering Protection l
7.2 How Can Developers Protect Apps from Reverse Engineering? li
7.2.1 Code Obfuscation (ProGuard / R8) li
7.2.2 String Encryption li
7.2.3 Root Detection lii
7.2.4 Debugger Detection lii
7.2.5 Tamper Detection lii
7.2.6 Native Code Protection lii
7.3 Mitigation Strategies liii
7.3.1 Secure Manifest Configuration liii
7.3.2 Service Permission Protection liii
7.3.3 Runtime Integrity Verification liii
7.3.4 Restrict Foreground Service Usage liv
7.3.5 Secure Coding Practices liv
7.3.6 Permission Minimization liv
8. Testing and Execution Evidence lvi
8.1 Build Success lvi
8.2 APK Output lvi
8.3 App Running Normally lvii
8.4 Notification Demo Execution lviii
8.5 Timed CPU Demo Execution lix
8.6 Logcat Verification lx
9. CONCLUSION lxii
APPENDIX A: INJECTED MALICIOUS SERVICE (DemoService.kt) lxiii
APPENDIX B: MODIFIED MAINACTIVITY TRIGGER LOGIC lxiv
APPENDIX C: MODIFIED AndroidManifest.xml lxvii

LIST OF FIGURES

Figure 1.3.1: Malware injection process into Android application
Figure 1.4.1 Amaze File Manager application architecture
Figure 1.5.1 Android application tampering and malware injection problem
Figure 3.2.1 Amaze APK Decompiled Structure

Figure 4.3.1 Android Emulator Running
Figure 5.1.1 Decompiled APK using JADX
Figure 5.1.2 Decompiled APK Main Structure
Figure 5.2.1.1 Activity code
Figure 5.2.2.1 Service code
Figure 5.2.3.1 Receiver code
Figure 5.2.4.1 Permission code identified
Figure 5.2.5.1 Identified Provider Code
Figure 5.2.6.1 Component Relationship
Figure 6.1.1.1 Decompiled apk structure using JADX
Figure 6.1.2.1 DemoService Code
Figure 6.1.3.1 Edited code in AndroidManifest.xml
Figure 6.1.4.1 Edited Code in MainActivity.java
Figure 6.1.5.1 Rebuild APK
Figure 6.2.1.1 DemoService.kt
Figure 6.2.2.1 DemoService registered in AndroidManifest.xml
Figure 6.2.3.1 Create 2 trigger options in the app menu
Figure 6.2.3.2 Starts DemoService in Notification mode
Figure 6.2.3.3 Time-Based CPU Trigger
Figure 6.4.1 All the Permission Codes in AndroidManifest.xml
Figure 6.5.1 Service Life Cycle Explanation
Figure 8.1.1 Successful Gradle build of the modified Amaze File Manager APK
Figure 8.2.1 Modified APK generated as AmazeFileManager_Modified.apk
Figure 8.3.1 Modified Amaze File Manager running normally in the emulator
Figure 8.4.1 Added menu trigger options visible in the modified application
Figure 8.4.2 Notification demo running as a foreground service
Figure 8.5.1 Timed CPU demo trigger activated on an even minute
Figure 8.5.2 CPU demo running as a foreground service
Figure 8.5.3 Timed CPU trigger blocked when the even-minute condition was not met
Figure 8.6.1 Logcat output confirming notification mode execution
Figure 8.6.2 Logcat output confirming CPU mode completion after approximately 10 seconds

LIST OF TABLES

Table 1.6.1 Scope of the Project
Table 1.6.2 Limitation of the Project
Table 2. Reverse Engineering Tasks and Purposes
Table 3.5.1 Tools used in this project and their purpose

Table 4.1.1 Hardware Components and Specifications
Table 4.2.1 Software and their Purposes
Table 4.3.1 Emulator's parameters and Values
Table 5.1.1 Folders of Decompiled APK and their purpose
Table 5.1.2 Important Files and their purpose
Table 5.1.2.1 Important Activities
Table 5.2.2.1 Services identified in AndroidManifest.xml
Table 5.2.3.1 Identified receiver code
Table 5.2.4.1 Permissions Identified
Table 5.2.5.1 Identified Provider Code
Table 5.4.3.1 Permissions and Risks
Table 6.2.1.1 Supported Modes
Table 6.2.2.1 Attributes and their Purpose
Table 6.5.2.1 Execution Modes

LIST OF ABBREVIATIONS

APK: Android Package Kit
API: Application Programming Interface
ADB: Android Debug Bridge
AOSP: Android Open Source Project
CPU: Central Processing Unit
DEX: Dalvik Executable
GUI: Graphical User Interface
IDE: Integrated Development Environment
INTENT: Android Messaging Object
JADX: Java Decompiler for Android
JAR: Java Archive
JSON: JavaScript Object Notation
KOTLIN: Modern Programming Language for Android
LOGCAT: Android Logging System
NDK: Native Development Kit
OS: Operating System
RAM: Random Access Memory
SDK: Software Development Kit
SMALI: Android Assembly Language
UI: User Interface
URI: Uniform Resource Identifier
XML: Extensible Markup Language

LIST OF APPENDICES

Appendix A: Injected Malicious Service (DemoService.kt)
Appendix B: Modified MainActivity Trigger Logic
Appendix C: Modified AndroidManifest.xml
Appendix D: Report Document & Resources

INTRODUCTION

1.1 Background of Android Application Security

The Android OS is currently one of the most popular operating systems in the world. Due to the openness and flexibility of this mobile operating system, it provides developers with numerous possibilities to create useful applications that interact directly with device hardware, storage, and other built-in capabilities. However, this makes the mobile operating system prone to various security issues. For instance, application permissions include access to storage and device information, networking, internet, background processes, and more. The possibility of malicious use of any of the above permissions cannot be excluded.

Android applications can be installed on the user's smartphone or tablet in the form of APK files. An APK file usually contains a set of files required for application execution, including compiled code and resources. Due to the relatively simple way to decompile Android applications, attackers can analyze the inner workings of an application and detect its vulnerabilities. Therefore, malicious parties can modify the source code of applications by exploiting existing vulnerabilities.

Another critical security issue with the Android operating system is the misuse of background services. An Android service enables applications to operate in the background without requiring any form of user intervention. Although this functionality can be used for productive purposes like file synchronization and background processing, background services can be exploited by hackers to silently carry out their nefarious tasks like collecting information about the mobile device, consuming its resources, or executing unauthorized code. The malicious background services can operate undetected by the end-user.

Moreover, most Android applications have permissions that are granted when they are installed on smartphone or tablet. In case there is any malicious code inserted in the legitimate application that already has extensive permissions, the malicious code can utilize these permissions for malicious reasons. Consequently, the likelihood of data theft, code execution, and resource consumption becomes more likely. Thus, it is vital to ensure that the Android application is protected from malicious attacks or reverse engineering.

The following project illustrates the process of reversing a legitimate Android application and adding a Trojan-like background service to it.

1.2 Reverse Engineering Overview

Reverse engineering can be defined as a technique where a compiled application is analyzed to know the inner workings of the software, the application functionalities and its behavior, without the use of source codes. When talking about reverse engineering on Android applications, then reverse engineering is done by decompiling APK files. Tools are needed to decompile APK files to get source codes, resources and configuration files.

Android APK contains many features such as AndroidManifest.xml, compiled source code, resources, and assets. Reverse engineering an Android APK provides details about what application activities can be accessed by users, application services, permissions and execution path of the application. From this information, one can determine whether there is any vulnerability in the app and where to inject malware in the application.

Also, reverse engineering provides a platform for the study of how various components work together in the application. Activities are responsible for controlling user interfaces, whereas services handle tasks in the background. Through the analysis of these components, one can learn how to call certain functions and change the application's behavior without interfering with its normal functioning. Reverse engineering is a widespread method used in malware injection and security research.

In the current project, reverse engineering was employed to perform a thorough investigation of the chosen application and reveal its architecture. The application's APK file was decompiled to study the AndroidManifest.xml file, application activities, services in the background, permissions, and flow control. Such analysis enabled the identification of the appropriate location where a Trojan-like service could be inserted.

The process of reverse engineering sets the stage for the subsequent insertion of the malicious code into the application. With knowledge of the application's architecture and flow control, one can incorporate the malicious code into the application and ensure that its normal functioning is not affected.

1.3 Malware Injection Concept

Malware injection is a method through which malicious code is added to an application program without any alteration in its original functions. It allows the application to perform some secret tasks without the user's knowledge. It involves modification of a genuine application and its usage in Trojanizing to carry out malicious tasks such as data acquisition, background execution, and resource usage.

Malware injection in Android application software is normally done after carrying out reverse engineering of the APK file. It involves the addition of other services to the application, as well as modifications of activities and configuration files within the application.

Malware injection was performed in this project by introducing a new background service called DemoService. This service acts like malware by gathering device information and performing an intensive loop on the processor and background tasks. This service was added in the AndroidManifest file, and the triggering was done via modifications done in the MainActivity.

The malware is based on a trigger mechanism. It means that the malicious payload is executed only under certain menu option selection from the user or after a time-based condition is met.

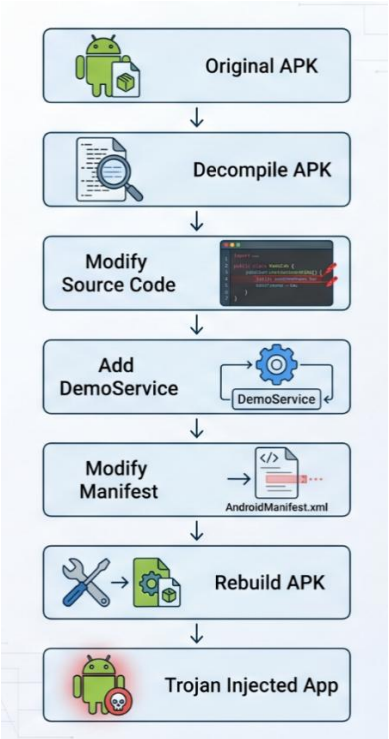

Figure 1.3.1 Malware injection process into Android application

1.4 Selected Application – Amaze File Manager

Amaze File Manager is an open-source Android file manager program that enables users to navigate through, manage, copy, move, and delete files stored in their devices. The app offers a user-friendly interface and implements file manipulation operations in the background by means of Android services. Because of the application's modular design and ability to perform background tasks, Amaze File Manager can be easily reverse-engineered and manipulated for malware demonstration purposes.

This app comprises several components such as activities, services, and resources. MainActivity.java is utilized for managing user interaction and navigation within the application. Additionally, there are background services used for performing asynchronous tasks like file copying and file compression.

The application also makes use of Android permission to access storage and system features on the phone. Such permissions give the app authority to perform operations related to file management but could be abused in case malicious code is introduced. This analysis helps identify how the application functions and where more functions can be embedded.

This paper presents a case where Amaze File Manager application is hacked and modified by injecting a malicious service. The malicious service is introduced as part of the services already present in the application. The malicious service is activated by manipulating the menu in MainActivity class.

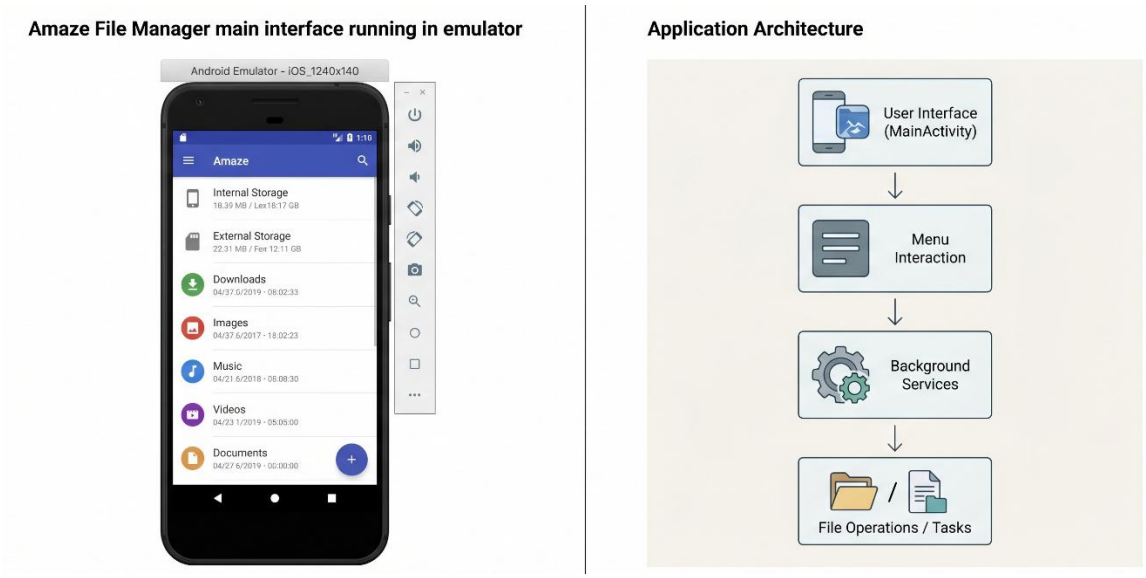

Figure 1.4.1 Amaze File Manager application architecture

1.5 Problem Statement

The applications for Android are usually distributed in the form of APKs. Such files can be easily decomposed and decompiled, with a variety of tools being freely available on the Internet. Consequently, it becomes possible to examine application design, change source code, and even include malware.

A critical problem with Android applications is that many lack integrity checks. As a result, once a program is modified, it will function properly but execute some other operations in the background. The possibilities include adding services, altering interface triggering events, or even introducing some conditional operations.

Another issue is permission abuse. In case an authentic application is granted certain storage or network permissions, it could be possible for a malicious program to abuse these permissions. Consequently, the malware will have the ability to work without seeking any extra permission from the user. This means that Trojan-injected applications can perform certain tasks using the permissions previously assigned.

The present research is aimed at tackling the issue of Android application injections by showcasing how malware can be successfully injected within an authentic application.

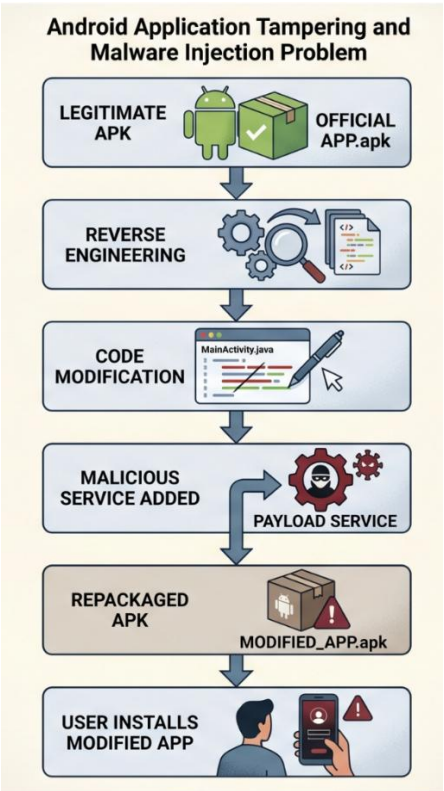

Figure 1.5.1 Android application tampering and malware injection problem

1.6 Scope and Limitations

This project aims to reverse engineer and inject a Trojan like service into an android app. This is to illustrate the malicious activity that can be done through such an app. In addition, it still functions like any other app in use.

Scope of the Project**:**

Area	**Description**
APK Reverse Engineering	Decompile Amaze File Manager APK
Manifest Analysis	Analyze activities, services and permissions
Component Mapping	Identify injection points
Malware Injection	Add DemoService background service
Trigger Implementation	Menu trigger + time-based trigger
Background Execution	Run CPU workload and logging
Testing Environment	Android Emulator
Verification	Logcat output analysis

Table 1.6.1 Scope of the Project

Limitation of the Project:

Limitation	**Description**
No real data theft	Only simulated device info logging
No external server	No real network exfiltration
Emulator only testing	Not tested on real devices
No system modification	Only application-level changes
Educational use only	Not for malicious deployment

Table 1.6.2 Limitation of the Project

2. Objectives

The purpose of this project is to show how a malicious component similar to a trojan can be injected into an Android app after reverse engineering and then examine the security threats involved. The objectives of this reverse engineering include studying application design, implementing malware injections, and performing execution testing.

Reverse engineering involves extracting information from the APK file regarding key components like AndroidManifest.xml, application activities, services, permissions, and other details related to application process flows.

Injection of malware involves designing a malware background service that will be used in the application. This injected malware background service will be able to simulate functions like collecting information from the device, executing processes in the CPU, and performing unauthorized background.

Goal of Security Analysis: To conduct a risk assessment on the injected component. This would be in the form of analyzing the misuse of permissions, backdoor operations, and triggers. The research work also determines the mitigation measures to prevent applications from such kinds of vulnerabilities.

Goal of Demonstration: To ensure that the injection process was executed successfully. In this case, the hacked application would be installed on the emulator and activated using Logcat output monitoring.

Reverse Engineering Tasks:

Task	**Purpose**
Decompile APK	Extract Readable Source Code
Analyze Manifest	Identify Components
Identify Activities	Understand UI Flow
Identify Services	Find Injection Point
Check permissions	Analyze Security Risks
Map Control Flow	Understand Execution

Table 2. Reverse Engineering Tasks and Purposes

Project objectives workflow:

Reverse Engineering
↓
Malware Injection
↓
Trigger Implementation
↓
Testing & Execution
↓
Security Analysis

3. Methodology

3.1 Overall Project Methodology

The overall methodology of this project consists of five major phases: application selection, reverse engineering, Trojan injection, testing, and security analysis. Each phase builds upon the previous phase to ensure systematic execution of the project.

The first phase involved selecting a suitable Android application for analysis. The selected application needed to contain background services, user interface triggers, and a modular structure. Amaze File Manager was chosen because it provides these characteristics and allows service-based execution.

The second phase involved reverse engineering applications. The APK file was decompiled using reverse engineering tools to extract the source code, manifest file, and resources. The structure of the application was analyzed to identify components such as activities, services, and UI elements.

The third phase involved Trojan injection. A malicious service was created and added to the application. The service was designed to simulate device information collection, CPU workload execution, and background activity. The application manifest was modified to register the malicious service.

The fourth phase involved modifying the user interface. New menu options were added to trigger the malicious service. Additionally, time-based conditional logic was implemented to simulate stealth activation.

The final phase involved testing and demonstration. The modified application was rebuilt and installed on an emulator. The Trojan service was triggered, and execution evidence was captured.

3.2 Reverse Engineering Method

The reverse engineering process began by obtaining the APK file of the selected application. The APK file was then decompiled to extract readable source code and configuration files. The decompiled project structure allowed analysis of application components and execution logic.

The reverse engineering method included:

1. Decompiling APK file
2. Extracting AndroidManifest.xml
3. Identifying activities and services
4. Analyzing permission usage
5. Studying application logic flow
6. Identifying injection points
7. Examining UI menu structure
8. Locating service architecture

The AndroidManifest file was analyzed to understand how the application components were registered. Activities were examined to understand UI flow, while services were analyzed to identify background execution capabilities. This analysis helped determine where the Trojan component could be inserted.

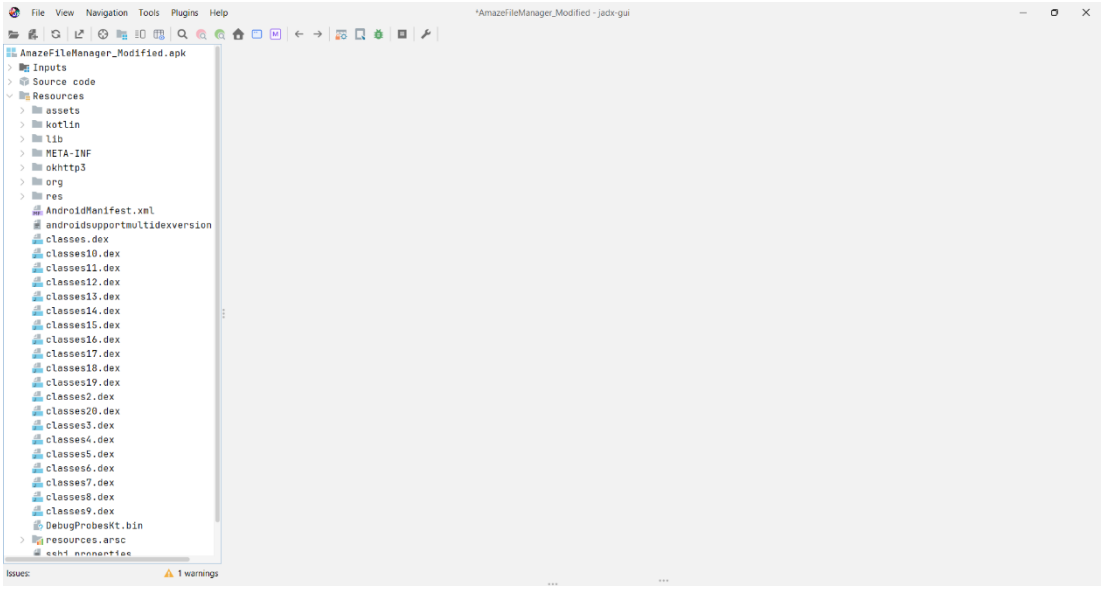

Figure 3.2.1 Amaze File Manager APK opened in JADX GUI

3.3 Malware Injection Method

This method involved creating a malicious service and integrating it into the reverse engineered application. The service was designed to simulate typical Trojan behavior such as background execution, device information logging, and CPU workload.

The injection method included the following steps:

1. Create DemoService.kt malicious service
2. Add service to project directory
3. Register service in AndroidManifest.xml

4. Modify MainActivity.java to add triggers
5. Implement trigger logic
6. Implement CPU workload

The DemoService.kt component collects device information such as manufacturer, model, Android version, SDK level, and brand. The service also executes a CPU-intensive loop for a fixed duration to simulate resource exhaustion. A notification demo was also implemented to simulate unauthorized background activity.

3.4 Testing Method

Testing was performed to verify that the injected Trojan service executes correctly. The modified application was rebuilt and installed on an emulator. The triggers were activated to confirm background execution.
Testing included:

- Installing modified APK
- Launching application
- Triggering notification demo
- Triggering timed CPU demo
- Monitoring Logcat output
- Observing CPU usage
- Verifying service execution
- Capturing screenshots

The Logcat tool was used to verify service execution and confirm device information logging. CPU usage was monitored to confirm that the CPU exhaustion payload executed successfully.

3.5 Tools and Technologies Used

Several tools were used to complete the reverse engineering and Trojan injection process. These tools were used for decompilation, editing, rebuilding, and testing the application.

Tools used in this project include:

Tool	**Purpose**
JADX GUI	APK Decompilation
Android Studio	Editing and testing APK
Java	Application Logic

Kotlin	DemoService.kt implementation
LogCat	Execution Verification

Table 3.5.1 Tools used in this project and their purpose

These tools allowed the application to be reverse engineered, modified, rebuilt, and tested in a controlled environment.

4. Development and Testing Environment

This chapter describes the hardware and software environment used to complete the project. It also explains the emulator configuration and tools used for reverse engineering, Trojan injection, and testing. The entire experiment was conducted in a controlled environment to ensure safe execution of the modified application.

4.1 Hardware Environment

The project was conducted using a personal computer capable of running Android emulation and reverse engineering tools. The hardware environment provided sufficient processing power and memory to decompile the application, modify source files, rebuild the APK, and test the modified application.

The hardware configuration used for this project is listed below:

Component	**Specification**
Operating System	Windows 11
Processor	Intel/AMD Multi-core Processor
RAM	8GB or Higher
Storage	Minimum 20GB free
Graphics	Integrated GPU
Input Devices	Keyboard and Mouse

Table 4.1.1 Hardware Components and Specifications

The Android emulator requires sufficient RAM and CPU resources to run smoothly. The CPU exhaustion payload implemented in the malicious service also required stable hardware performance for demonstration purposes.

4.2 Software Environment

The software environment consisted of the operating system, development tools, reverse engineering tools, and emulator platform used to execute the project. These tools were used to decompile, modify, rebuild, and test the Android application.

The software environment used in this project includes:

Software	**Purpose**
JADX GUI	Decompile APK
Android Studio	Modify decompiled files
Android Emulator	Run modified app
Java	Application logic

Kotlin	Trojan service
Android SDK	Build support
Logcat	Monitor execution

Table 4.2.1 Software and their Purposes

These tools were used to perform reverse engineering, inject malicious code, rebuild the APK, and test the modified application safely.

4.3 Emulator Setup

The modified application was tested using an Android Virtual Device (AVD) configured in Android Studio. The emulator simulates a Google Pixel 2 device and provides a controlled environment for executing the injected Trojan service.

The emulator configuration used in this project is shown below:

Parameter	**Value**
Device Model	Pixel 2
Manufacturer	Google
Android Version	Android 11
API Level	30
Resolution	1080 × 1920
Density	420 dpi
RAM	2048 MB
CPU Cores	4
ABI	x86_64
Storage	4243 MB
GPU	Enabled (Auto)
Network	Full speed
Play Store	Enabled

Table 4.3.1 Emulator's parameters and Values

The emulator was used to install the modified application and test the malicious service execution. The notification demo trigger and timed CPU demo trigger were executed inside the emulator. Logcat output was monitored to confirm service execution and device information logging.

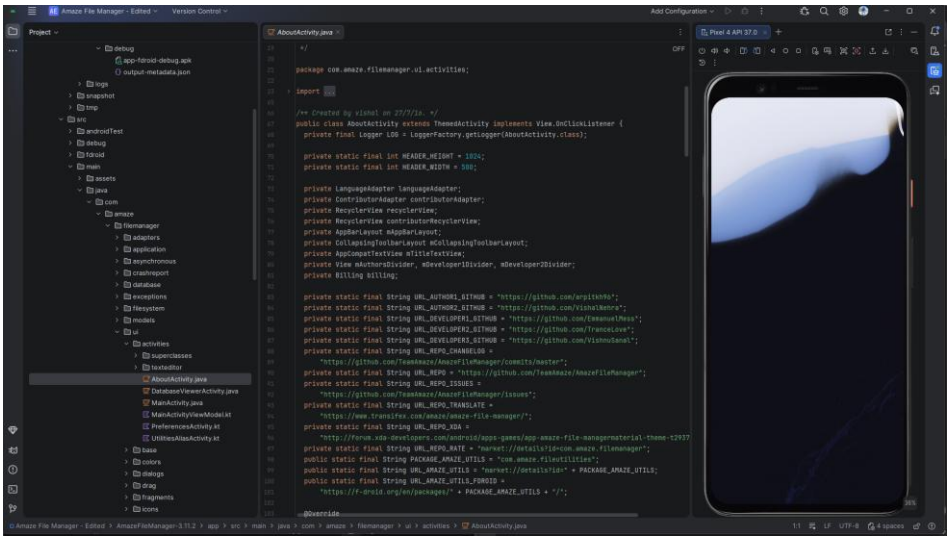

Figure 4.3.1 Android Emulator Running

5. Reverse Engineering Analysis

This chapter presents the reverse engineering analysis of the selected Android application. The APK file was decompiled and examined to understand the internal structure, application components, permissions, network communication, hardcoded logic, and execution flow. The analysis helped identify suitable injection points for adding the malicious component.

5.1 APK Structure Analysis

Reverse engineering was done on the Amaze File Manager APK in order to understand its architecture and flow. Reverse engineering allowed us to decompile the APK through JADX, making it possible for us to obtain source code, resources, and configuration file details. From there, we were able to find appropriate places to inject our malicious service.

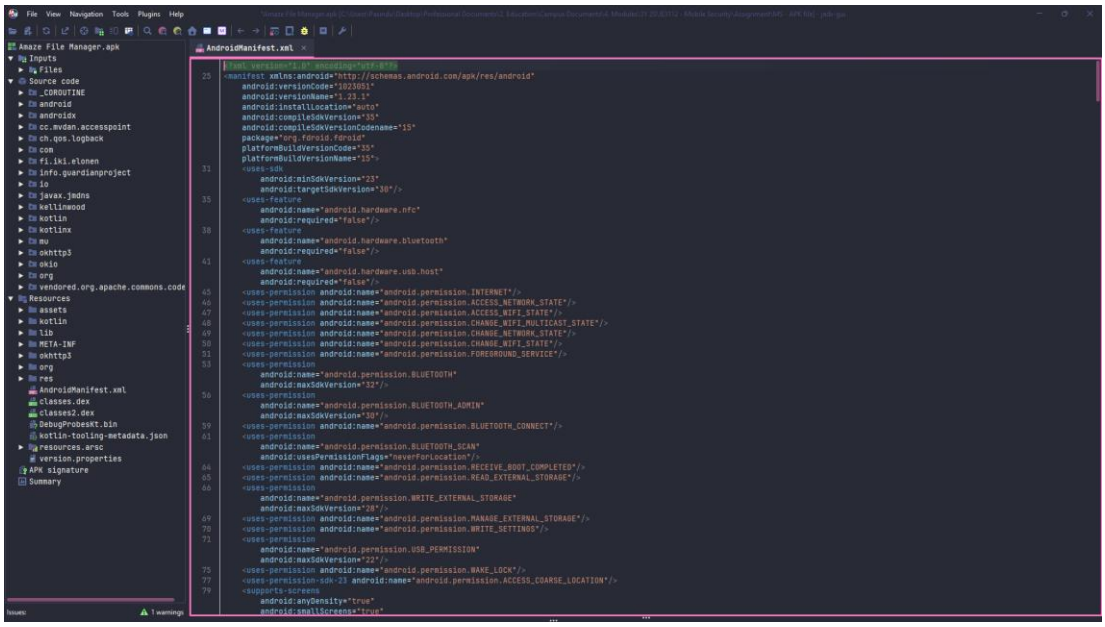

Figure 5.1.1 Decompiled APK using JADX

Decompiled APK Structure:

Folder	Purpose
resources	application configuration files
sources	decompiled Java/Kotlin code
assets	bundled configuration files
META-INF	signature and metadata
kotlin	kotlin runtime dependencies

Table 5.1.1 Folders of Decompiled APK and their purpose

The decompiled APK structure represents the extracted contents obtained

after reverse engineering the Amaze File Manager APK using JADX. The structure contains resources, metadata, assets, and compiled application components. The decompiled directory includes the AndroidManifest file, Kotlin libraries, META-INF metadata, and application resources.

```
Decompiled APK
│
├── resources
│   ├── AndroidManifest.xml
│   ├── assets
│   ├── kotlin
│   ├── META-INF
│   └── properties files
│
├── sources
│   └── com.amaze.filemanager
│
└── libraries
```

Figure 5.1.2 Decompiled APK Main Structure

Important Files Identified:

File	Purpose
AndroidManifest.xml	Application configuration
MainActivity.java	UI entry point
services package	background services
resources folder	UI layouts
values folder	app configuration

Table 5.1.2 Important Files and their purpose

5.2 Component Mapping

Component mapping was carried out to analyze the major application components and the location where the Malware could be injected. In the case at hand, the focus of the analysis was the Android Manifest file, activities, services, and the new components added to the modified application.

The manifest analysis revealed that the application contains 25 Activities, 16 Services, and 16 Broadcast Receivers, indicating complex component-based architecture suitable for background execution and trigger-based operations.

5.2.1 Activities Mapping

The AndroidManifest.xml defines 25 activities responsible for UI navigation and user interaction. The MainActivity was identified as the primary entry point and used to trigger the injected malicious service.

Important Activities Identified:

Activity	Purpose	Exported
MainActivity	Application entry point	true
AppDetailsActivity	Displays app details	true
ManageReposActivity	Repository management	false
InstallConfirmActivity	Install confirmation	false
CalculatorActivity	Hidden calculator mode	true
PanicResponderActivity	Panic trigger handler	true
InstalledAppsActivity	Installed apps list	false
CrashReportDialog	Crash reporting	false

Table 5.1.2.1 Important Activities

Ex:

```
<activity
   android:exported="true"
   android:label="@string/appbar_name"
   android:launchMode="singleInstance"
   android:name=".ui.activities.MainActivity"
   android:theme="@style/appCompatBlack"
   android:configChanges="uiMode"
   android:windowSoftInputMode="adjustPan">
```

When an activity is marked as *android:exported="true"*, it means:

- Other apps can launch this activity
- The system launcher can start it
- It can receive external intents

This makes it publicly accessible and easier to use as a trigger point.

```
<activity
    android:exported="true"
    android:label="@string/appbar_name"
    android:launchMode="singleInstance"
    android:name=".ui.activities.MainActivity"
    android:theme="@style/appCompatBlack"
    android:configChanges="uiMode"
    android:windowSoftInputMode="adjustPan"
  >
```

Figure 5.2.1.1 Activity code

5.2.2 Service Mapping

The application contains 16 background services used for downloading, installing, scanning, and background execution. These services run without direct user interaction, making them ideal injection targets.

Services Identified:

Service	Purpose
.asynchronous.services.ExtractService	File extraction
.asynchronous.services.ZipService	File compression
.asynchronous.services.CopyService	File copying
.asynchronous.services.EncryptService	File encryption
.asynchronous.services.DecryptService	File decryption
.asynchronous.services.ftp.FtpService	FTP server
.asynchronous.services.ftp.FtpTileService	Quick settings FTP tile

Table 5.2.2.1 Services identified in AndroidManifest.xml

Ex:

<service android:name=".asynchronous.services.CopyService"
android:label="Copying"
android:foregroundServiceType="dataSync" />

```
<service android:name=".asynchronous.services.ExtractService"
    android:label="Extracting"
    android:foregroundServiceType="dataSync" />

<service android:name=".asynchronous.services.ZipService"
    android:label="Compressing"
    android:foregroundServiceType="dataSync" />

<service android:name=".asynchronous.services.CopyService"
    android:label="Copying"
    android:foregroundServiceType="dataSync" />

<service android:name=".asynchronous.services.EncryptService"
    android:label="@string/crypt_encrypting"
    android:foregroundServiceType="dataSync" />
```

Figure 5.2.2.1 Service code

5.2.3 Broadcast Receiver Mapping

The manifest defines a broadcast receiver used for controlling the FTP background service.

Receiver Identified:

Receiver	Purpose
.asynchronous.services.ftp.FtpReceiver	Start/Stop FTP service

Table 5.2.3.1 Identified receiver code

Ex:

<receiver
android:name=".asynchronous.services.ftp.FtpReceiver"
android:exported="true"

android:permission="${applicationId}.permission.CONTROL_FTP_SERVER">
<intent-filter>
<action
android:name="com.amaze.filemanager.services.ftpservice.FTPReceiver.ACTION_START_FTPSERVER" />
<action
android:name="com.amaze.filemanager.services.ftpservice.FTPReceiver.ACTION_STOP_FTPSERVER" />
</intent-filter>
</receiver>

```
<receiver
    android:name=".asynchronous.services.ftp.FtpReceiver"
    android:exported="true"
    android:permission="${applicationId}.permission.CONTROL_FTP_SERVER">
    <intent-filter>
        <action android:name="com.amaze.filemanager.services.ftpservice.FTPReceiver.ACTION_START_FTPSERVER" />
        <action android:name="com.amaze.filemanager.services.ftpservice.FTPReceiver.ACTION_STOP_FTPSERVER" />
    </intent-filter>
</receiver>
```

Figure 5.2.3.1 Receiver codes

5.2.4 Permission Mapping

The AndroidManifest.xml requests multiple permissions that allow storage access, network communication, and background execution.

Critical Permissions Identified:

Permission	**Purpose**	**Security Risk**
INTERNET	Network communication	Data exfiltration
ACCESS_NETWORK_STATE	Network status	Reconnaissance
ACCESS_WIFI_STATE	WiFi information	Network tracking
READ_EXTERNAL_STORAGE	Read files	Data theft
WRITE_EXTERNAL_STORAGE	Modify files	Data tampering
MANAGE_EXTERNAL_STORAGE	Full file system access	Full compromise
FOREGROUND_SERVICE	Background execution	Persistent malware
WAKE_LOCK	Prevent sleep	CPU abuse
REQUEST_INSTALL_PACKAGES	Install apps	Malware dropper
REQUEST_DELETE_PACKAGES	Remove apps	Security bypass
QUERY_ALL_PACKAGES	List installed apps	Reconnaissance
POST_NOTIFICATIONS	Show notifications	Phishing alerts

Table 5.2.4.1 Permissions Identified

Ex:

<uses-permission android:name="android.permission.WAKE_LOCK" />

<uses-permission android:name="android.permission.ACCESS_WIFI_STATE" />

```
<uses-permission android:name="android.permission.WAKE_LOCK" />
<uses-permission android:name="android.permission.ACCESS_WIFI_STATE" />
<uses-permission android:name="android.permission.INTERNET" />
<uses-permission android:name="android.permission.CHANGE_WIFI_MULTICAST_STATE" />
<uses-permission android:name="android.permission.NEARBY_WIFI_DEVICES"
    android:usesPermissionFlags="neverForLocation"
    tools:targetApi="33" />
<uses-permission android:name="android.permission.ACCESS_NETWORK_STATE" />
<uses-permission android:name="android.permission.READ_EXTERNAL_STORAGE" />
<uses-permission android:name="android.permission.WRITE_EXTERNAL_STORAGE"/>
<uses-permission android:name="com.android.launcher.permission.INSTALL_SHORTCUT" />
<uses-permission android:name="com.amaze.cloud.permission.ACCESS_PROVIDER" />
<uses-permission android:name="android.permission.USE_FINGERPRINT" />
<uses-permission android:name="android.permission.REQUEST_INSTALL_PACKAGES" />
<uses-permission android:name="android.permission.FOREGROUND_SERVICE" />
<uses-permission android:name="android.permission.REQUEST_DELETE_PACKAGES" />
<uses-permission android:name="android.permission.MANAGE_EXTERNAL_STORAGE" />
<uses-permission android:name="android.permission.QUERY_ALL_PACKAGES" />
<uses-permission android:name="android.permission.POST_NOTIFICATIONS" />
<uses-permission android:name="android.permission.FOREGROUND_SERVICE_DATA_SYNC" />
```

Figure 5.2.4.1 Permission code identified

5.2.5 Providers

The application also defines a FileProvider for sharing files securely.

Provider	Purpose
androidx.core.content.FileProvider	Secure file sharing

Table 5.2.5.1 Identified Provider Code

Ex:

```
<provider
        android:name="androidx.core.content.FileProvider"
        android:authorities="${applicationId}"
        android:exported="false"
        android:grantUriPermissions="true">
        <meta-data
            android:name="android.support.FILE_PROVIDER_PATHS"
            android:resource="@xml/provider_paths" />
    </provider>
```

```
<provider
    android:name="androidx.core.content.FileProvider"
    android:authorities="${applicationId}"
    android:exported="false"
    android:grantUriPermissions="true">
    <meta-data
        android:name="android.support.FILE_PROVIDER_PATHS"
        android:resource="@xml/provider_paths" />
</provider>
```

Figure 5.2.5.1 Identified Provider Code

5.2.6 Component Relationship

The interaction between components is as follows:

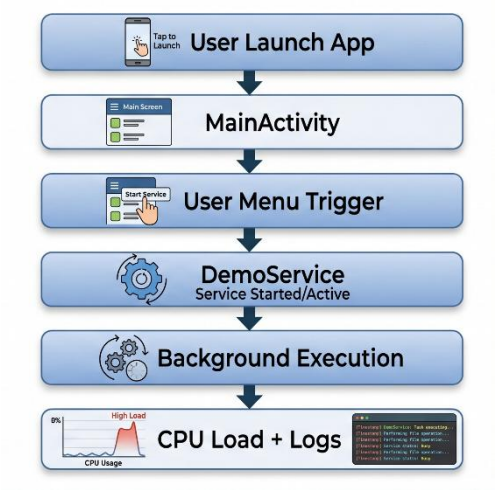

Figure 5.2.6.1 Component Relationship

5.3 Control Flow Explanation

Control Flow Analysis was carried out to gain an understanding of how the program would flow from when it started running until the point at which it would start the background service. From the analysis, it became clear that the malicious service could be introduced without disturbing the application flow.

5.3.1 Application Entry Flow

The application starts from MainActivity, which is defined as the launcher activity in the AndroidManifest.xml. This activity loads the main file manager interface and handles user interaction.

App Launch
↓
MainActivity.xml
↓
UI Loaded
↓
User Interaction

5.3.2 Activity Flow

After launching MainActivity, users interact with menu and file operations. These interactions call internal methods that start background services.

MainActivity
↓
User selects operation
↓
File operation requested
↓
Start Background Service

5.3.3 Service Execution Flow

The application uses asynchronous services for long-running tasks such as copy, zip, and extract operations.

MainActivity
↓
User Action
↓
CopyService / ZipService / ExtractService
↓
Foreground Service
↓
Background Execution

5.4 Identified Vulnerabilities

There have been found various security flaws through the reverse engineering of Amaze File Manager app. The security issues that were identified allow hackers to change the application, insert malicious service, and even perform background work.

5.4.1 Exported Activity Vulnerabilities

The MainActivity is marked as exported, which allows external applications or modified code to trigger it.

Manifest Evidence:

```
<activity
android:name=".ui.activities.MainActivity"
android:exported="true">
```

Security Risk:

- External apps can launch MainActivity
- Malicious triggers can be injected
- Service execution can start silently

This made MainActivity an ideal location to trigger the malicious DemoService.

5.4.2 Background Service Execution Vulnerability

The application uses multiple foreground services for background file operations. These services run without direct user interaction.

Ex. Service:

```
<service
android:name=".asynchronous.services.CopyService"
android:foregroundServiceType="dataSync"/>
```

Security Risk:

- Long running background execution
- Hidden malicious processing
- CPU abuse possibility
- Data collection in background

The injected DemoService used this same mechanism.

5.4.3 Excessive Permission Usage

The application requests powerful permissions that allow full device access.

High Risk Permissions:

Permission	**Risk**
MANAGE_EXTERNAL_STORAGE	Full file system access
INTERNET	Data exfiltration
WAKE_LOCK	CPU abuse
QUERY_ALL_PACKAGES	App reconnaissance
REQUEST_INSTALL_PACKAGES	Malware dropper
FOREGROUND_SERVICE	Persistent execution

Table 5.4.3.1 Permissions and Risks

These permission increase attack surface.

6. Malware Injection Design

6.1 Step by Step Injection Process

The malware injection process was performed after decompiling the Amaze File Manager APK. The malicious component was added as a background service and triggered from the main activity.

The injection process consists of the following steps:

1. Decompile APK
2. Add malicious service file
3. Modify AndroidManifest.xml
4. Modify MainActivity trigger
5. Rebuild APK
6. Install modified APK
7. Execute trigger

6.1.1 Decompile APK

The APK file was decompiled using JADX GUI to obtain readable source code and resources.

Tool Used: JADX GUI 1.5.5

Decompiled Structure:

resources/

sources/

Full Structure: AFM-D.md

This allowed access to:

- AndroidManifest.xml
- Java source files
- Kotlin files
- Activities
- Services

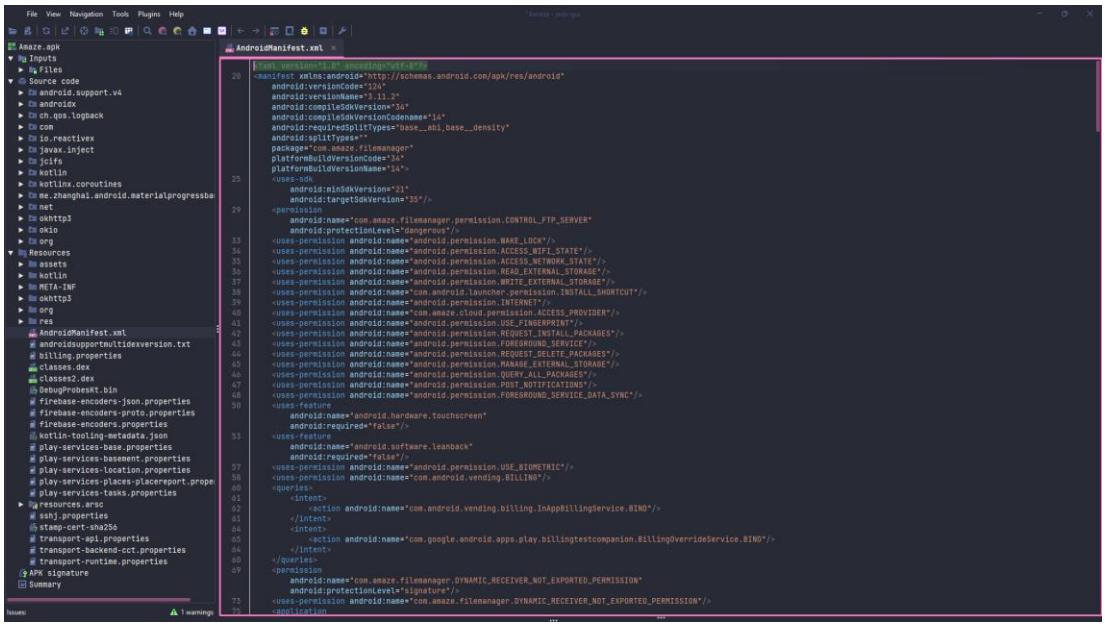

Figure 6.1.1.1 Decompiled apk structure using JADX

6.1.2 Add Trojan Service

The DemoService.kt file was added into the files.

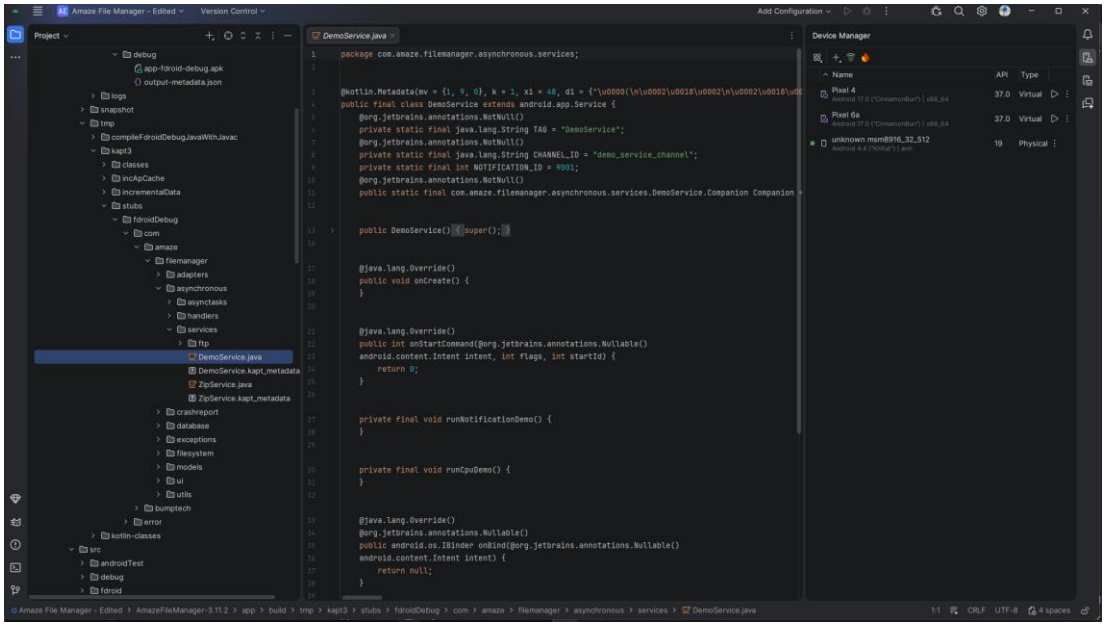

Figure 6.1.2.1 DemoService Code

Purpose of DemoService:

1. Collect device information
2. Generate CPU usage Spike
3. Unauthorized Notification Generation
4. Log execution

This service behaves like a Trojan payload.

6.1.3 Modify AndroidManifest.xml

The simulated service was registered in AndroidManifest.xml.

Added Service Entry:

```
<service
android:name=".asynchronous.services.DemoService"
android:foregroundServiceType="dataSync"
android:exported="false"/>
```

This allows Android to start the malicious service.

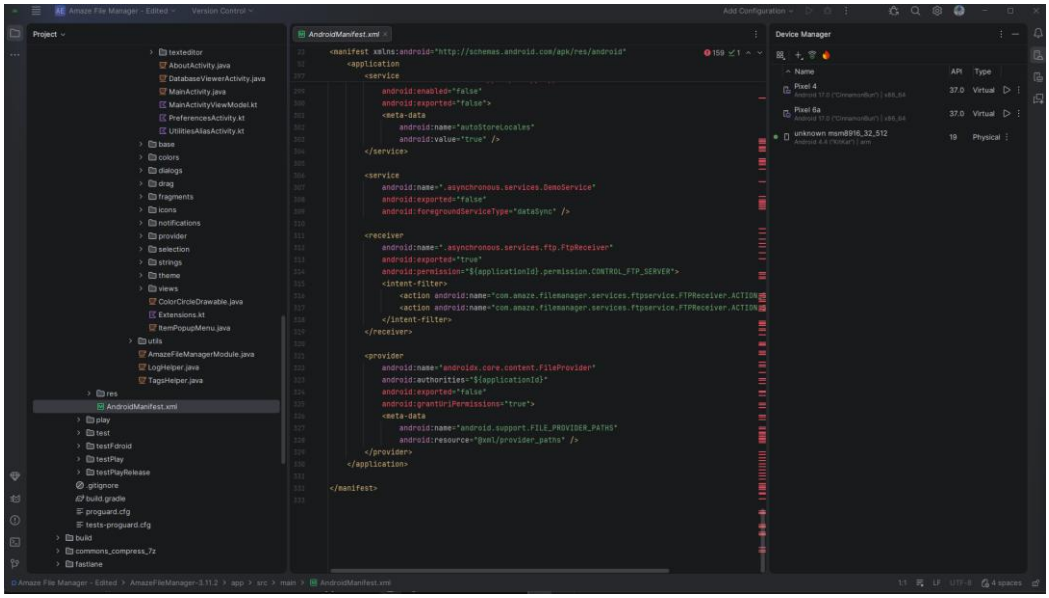

Figure 6.1.3.1 Edited code in AndroidManifest.xml

6.1.4 Modify MainActivity (Trigger)

The malicious service is triggered from the onOptionsItemSelected() method inside MainActivity. This method handles menu item selections and starts the injected DemoService based on user interaction or time-based conditions..

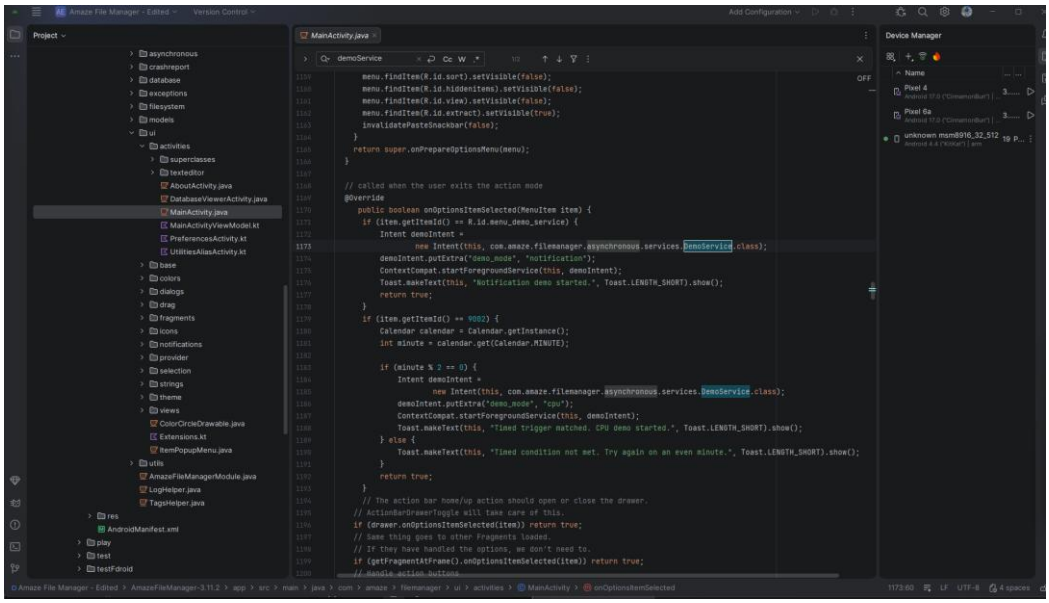

Figure 6.1.4.1 Edited Code in MainActivity.java

6.1.5 Rebuild APK

After modifications, the APK was rebuilt using Android Studio.

Steps:

- Save modified files
- Rebuild project
- Generate modified debug APK

Output:

AmazeFileManager_Modified.apk

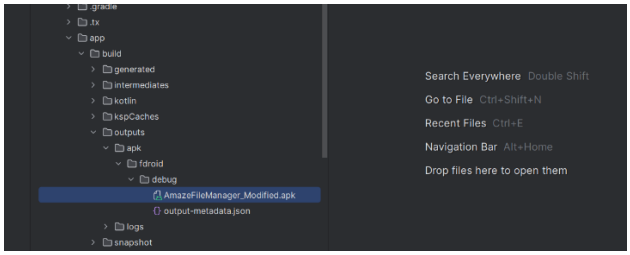

Figure 6.1.5.1 Rebuild APK

6.1.6 Install Modified APK

The modified APK was installed on Android emulator.

Steps:

- Drag APK to emulator
- Install
- Launch app

This installs or updates the debug version of the modified application in the emulator.

6.2 Code Modification with Explanation

This section explains the code changes made to inject the malicious Trojan service into the Amaze File Manager application. Three files were modified:

1. DemoService.kt (New malicious service)
2. AndroidManifest.xml (Service registration)
3. MainActivity.java (Trigger logic)

6.2.1 DemoService.kt (Trojan Service)

A new service named DemoService.kt was added to simulate malicious background behavior. This service collects device information, generates CPU load, and runs in foreground mode.

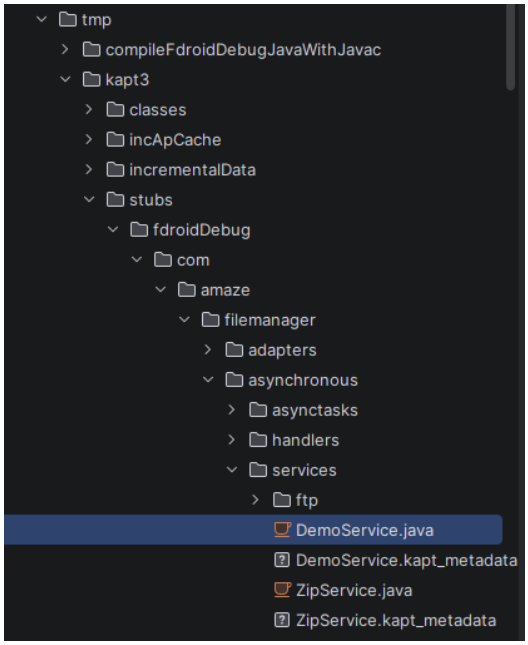

Figure 6.2.1.1 DemoService.kt

6.2.1.1 Unauthorized Notification Execution

The injected service creates and displays a foreground notification without explicit user consent. This allows the malicious service to run persistently in the background.

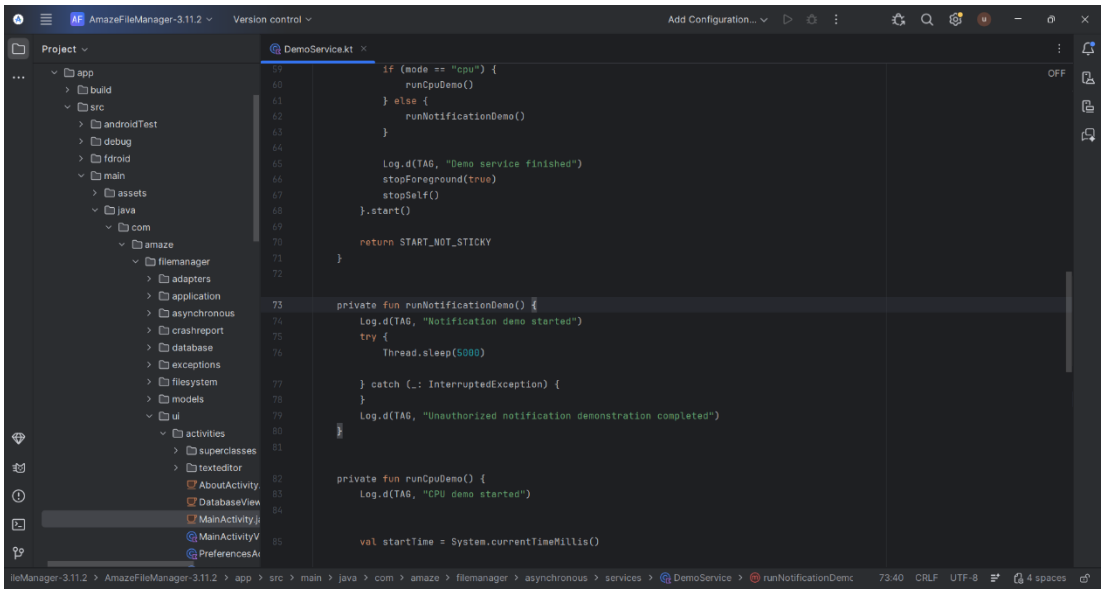

Figure 6.2.1.1.1 Notification Trigger in DemoService.kt

This Code performs following actions:

1. Creates a notification builder
2. Sets Title and Description
3. Assign notification icon
4. Build Notification object
5. Starts foreground service

Results:

1. Controlled foreground service execution
2. Service priority increase
3. Stealth background activity

6.2.1.2 Device Information Collection

The service collects device metadata using Android Build properties. This simulates information harvesting.

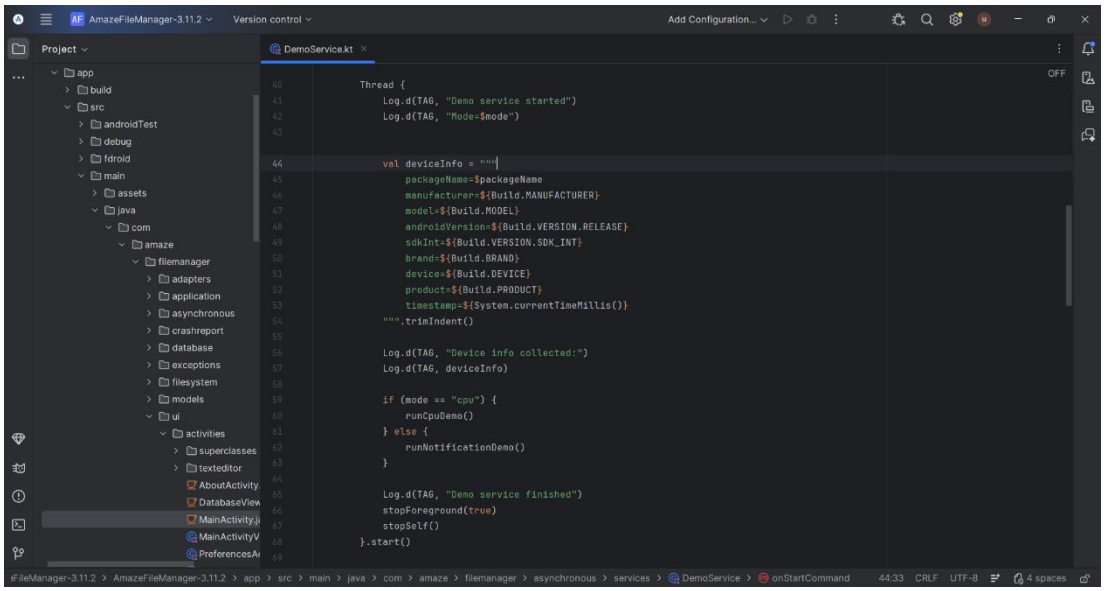

Figure 6.2.1.2.1 Device Info Collection Code

This Code Collects:

1. Application Package Name
2. Device Manufacturer
3. Device model
4. Android version
5. SDK version
6. Device brand
7. Device name
8. Product name
9. Current timestamp

6.2.1.3 CPU Spike Simulation

The malicious service generates a CPU spike by running a loop continuously for a fixed duration.

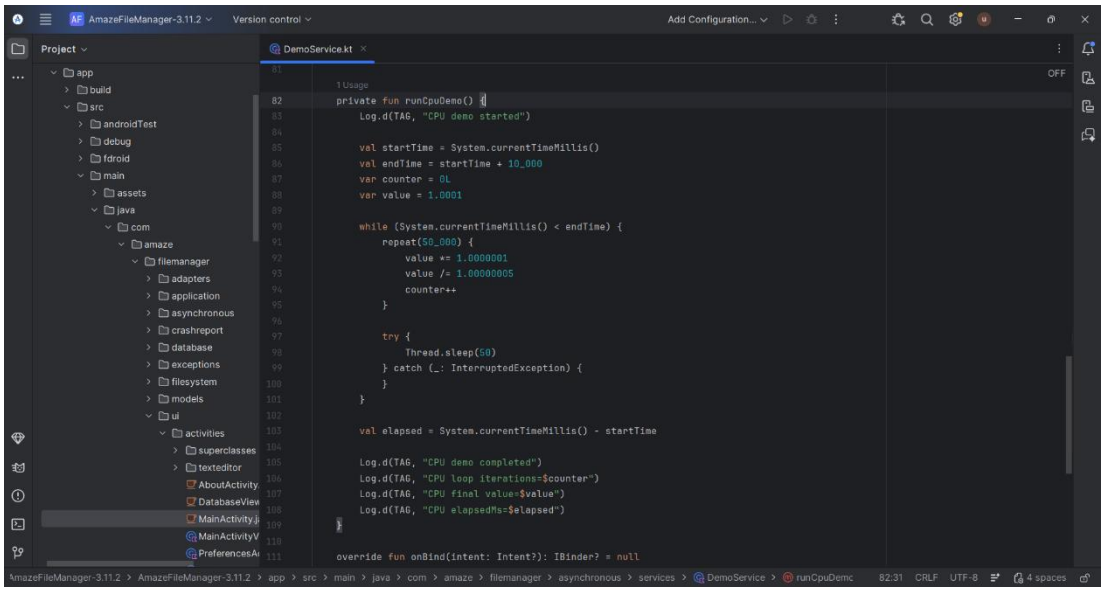

Figure 6.2.1.3.1 CPU Spike Trigger in DemoService.kt

This code performs:

1. Stores start time
2. Enters continuous loop
3. Executes heavy calculation
4. Runs for 10 seconds
5. Exits loop

This can result in:

1. Temporary CPU usage increase
2. Short-term battery impact
3. Temporary performance slowdown
4. Controlled background workload

6.2.2 MainActivity.java Modification

The MainActivity.java file was modified to trigger the malicious DemoService. The activity does not directly perform the payloads; instead, it starts DemoService with different modes, which then executes:

1. Unauthorized notification
2. Device information collection
3. CPU spike

6.2.2.1 Unauthorized Notification Trigger

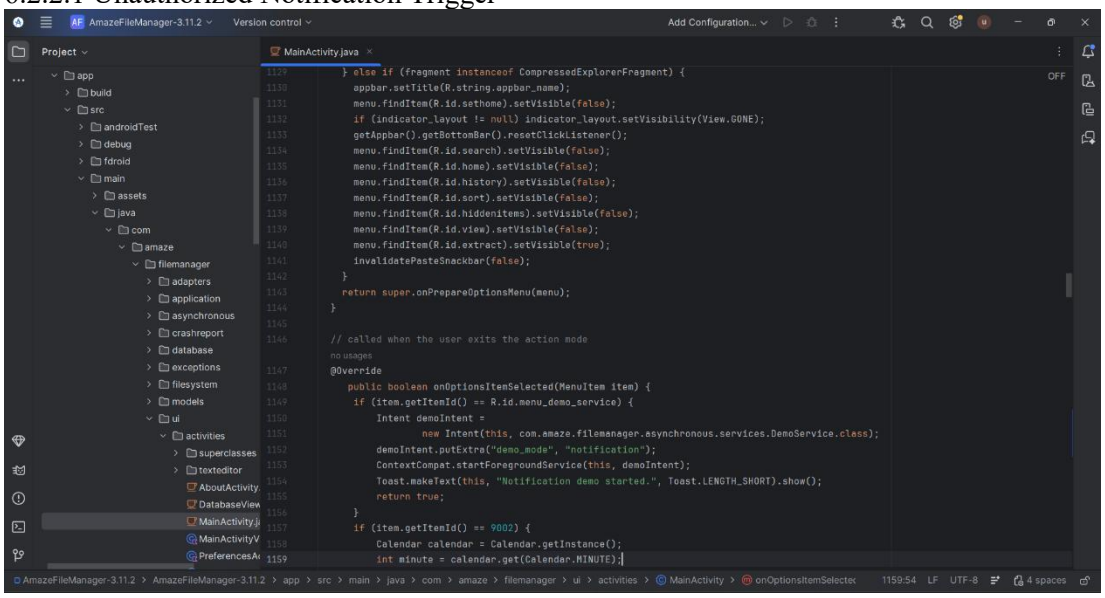

Figure 6.2.2.1.1 Notification Trigger

This code performs the following:

1. Checks menu item selection
2. Creates intent for DemoService
3. Passes "notification" mode
4. Starts foreground service
5. Shows toast message

Execution Flow:

User Click Menu > MainActivity Detects Selection > demo_mode = notification > DemoService started > Unauthorized notification shown

6.2.2.2 Device Information Collection Trigger

Device information collection is also triggered using the same intent.

Device information logging is performed when DemoService starts, before selecting the payload mode. Therefore, the metadata logging occurs in both notification mode and CPU mode.

Code: *demoIntent.putExtra("demo_mode", "notification");*

This parameter:

1. tells DemoService which payload to run
2. activates notification mode
3. triggers device info collection
4. logs device metadata

6.2.2.3 CPU Spike Trigger

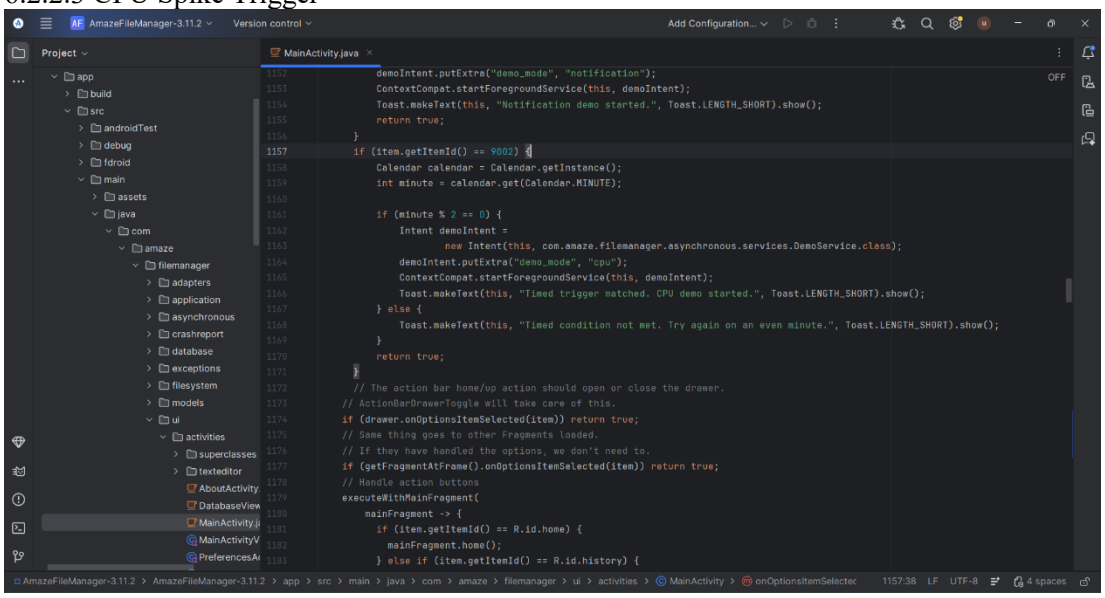

Figure 6.2.2.3.1 CPU trigger code

This code performs:

1. Checks menu item
2. Gets current time
3. Extracts minute value
4. Checks even minute condition
5. Starts service in CPU mode
6. Shows toast message

6.2.3 AndroidManifest.xml Modification

The malicious service was registered inside AndroidManifest.xml so Android OS can execute it.

Added Service Entry

```
<service
android:name=".asynchronous.services.DemoService"
android:foregroundServiceType="dataSync"
android:exported="false"/>
```

```
<service
    android:name=".asynchronous.services.DemoService"
    android:exported="false"
    android:foregroundServiceType="dataSync" />
```

Figure 6.2.2.1 DemoService registered in AndroidManifest.xml

Explanation:

Attribute	Purpose
name	DemoService location
foregroundServiceType	background execution
exported	internal service only

Table 6.2.2.1 Attributes and their Purpose

This allows the injected service to run in background.

6.3 Trigger Logic Implementation

The injected Trojan service is triggered by MainActivity using menu-based user interaction and a time-based conditional check. Two activation mechanisms were implemented to demonstrate real malware execution.

6.3.1 Notification Trigger (User Interaction)

The first trigger activates the malicious service when the user selects Run Notification Demo from the menu.

Trigger Code:

```
if (item.getItemId() == R.id.menu_demo_service) {
   Intent demoIntent =
         new Intent(this,
com.amaze.filemanager.asynchronous.services.DemoService.class);
   demoIntent.putExtra("demo_mode", "notification");
   ContextCompat.startForegroundService(this, demoIntent);
}
```

Execution Flow:

User opens menu
↓
Select "Run Notification Demo"
↓
MainActivity
↓
Start DemoService
↓
Notification payload

This trigger demonstrates user interaction-based activation.

6.3.2 CPU Usage Spike Trigger

The second trigger activates the payload only when the system minute is even.

Trigger Code:

```
Calendar calendar = Calendar.getInstance();
int minute = calendar.get(Calendar.MINUTE);
if (minute % 2 == 0) {
   demoIntent.putExtra("demo_mode", "cpu");
   ContextCompat.startForegroundService(this, demoIntent);
}
```

Execution Flow:

User selects CPU demo
↓
Check system time
↓
Even minute ?
YES
↓
Start DemoService
↓
CPU workload

This trigger demonstrates conditional stealth activation.

6.3.3 Payload Selection

The trigger passes a parameter to DemoService.

```
demoIntent.putExtra("demo_mode", "notification");
demoIntent.putExtra("demo_mode", "cpu");
```

DemoService reads the mode:

notification → show notification
cpu → run CPU workload

This allows multiple malicious behaviors using one service.

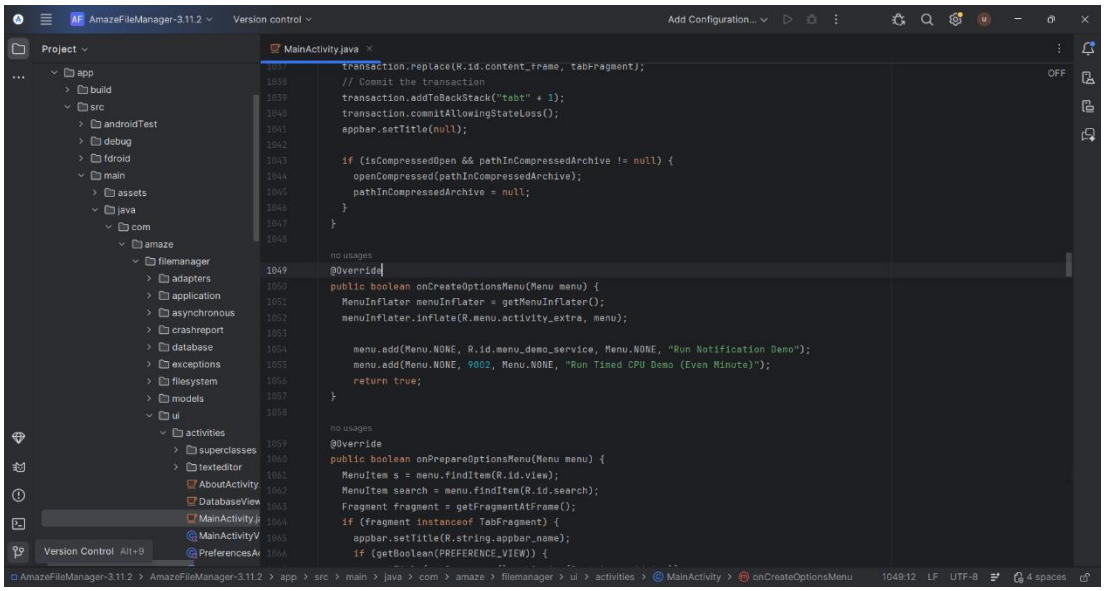

Figure 6.3.3.1 Payload Menu Selection

6.4 Permission Modification

To support the injected malicious service, the application uses permissions that allow background execution, notifications, and system resource access. These permissions enable the Trojan service to run persistently and perform background operations.

The permissions were analyzed from the modified AndroidManifest.xml.

```
<permission android:name="${applicationId}.permission.CONTROL_FTP_SERVER"
    android:protectionLevel="dangerous" />

<uses-permission android:name="android.permission.WAKE_LOCK" />
<uses-permission android:name="android.permission.ACCESS_WIFI_STATE" />
<uses-permission android:name="android.permission.ACCESS_NETWORK_STATE" />
<uses-permission android:name="android.permission.READ_EXTERNAL_STORAGE" />
<uses-permission android:name="android.permission.WRITE_EXTERNAL_STORAGE"/>
<uses-permission android:name="com.android.launcher.permission.INSTALL_SHORTCUT" />
<uses-permission android:name="android.permission.INTERNET" />
<uses-permission android:name="com.amaze.cloud.permission.ACCESS_PROVIDER" />
<uses-permission android:name="android.permission.USE_FINGERPRINT" />
<uses-permission android:name="android.permission.REQUEST_INSTALL_PACKAGES" />
<uses-permission android:name="android.permission.FOREGROUND_SERVICE" />
<uses-permission android:name="android.permission.REQUEST_DELETE_PACKAGES" />
<uses-permission android:name="android.permission.MANAGE_EXTERNAL_STORAGE" />
<uses-permission android:name="android.permission.QUERY_ALL_PACKAGES" />
<uses-permission android:name="android.permission.POST_NOTIFICATIONS" />
<uses-permission android:name="android.permission.FOREGROUND_SERVICE_DATA_SYNC" />
```

Figure 6.4.1 All the Permission Codes in AndroidManifest.xml

6.4.1 Foreground Service Permission

The injected DemoService runs as a foreground service. This requires

foreground service permission.
Manifest Permission:

<uses-permission android:name="android.permission.FOREGROUND_SERVICE"/>

Purpose:

- Allow long running background service
- Prevent Android from killing service
- Enable persistent execution

Security Impact:

- Self-terminating simulated payload
- Temporary foreground execution window
- Resource abuse

6.4.2 Foreground Data Sync Permission

The service uses foreground service type dataSync.

<uses-permission android:name="android.permission.FOREGROUND_SERVICE_DATA_SYNC"/>

Purpose:

- Allow dataSync foreground service
- Background processing tasks
- Long running operations

Security Risk:

- Hidden CPU intensive operations
- Silent background execution

6.4.3 Notification Permission

The malicious service displays a foreground notification.

<uses-permission android:name="android.permission.POST_NOTIFICATIONS"/>

Purpose:

- Show foreground notification
- Keep service alive
- Simulate legitimate background task

Security Risk:

- Fake system notifications
- Social engineering alerts
- Hidden malicious execution

6.4.4 Wake Lock Permission

The application uses wake lock to prevent device sleep during execution.

<uses-permission android:name="android.permission.WAKE_LOCK"/>

Purpose:

- Keep CPU active
- Prevent sleep
- Ensure payload completes

Security Risk:

- Battery drain
- CPU abuse
- Hidden background workload

6.5 Service Lifecycle Explanation

The injected DemoService follows the Android service lifecycle. The service is started from MainActivity, runs in foreground mode, executes the payload, and stops after completion.

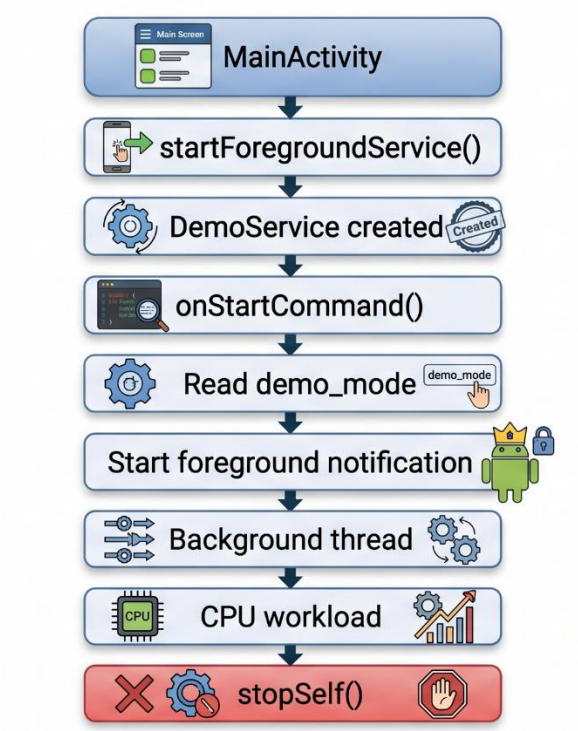

Figure 6.5.1 Service Life Cycle Explanation

6.5.1 Service Start (onStartCommand)

The service begins execution when MainActivity calls:

ContextCompat.startForegroundService(this, demoIntent);

This starts the DemoService and triggers the lifecycle method:

override fun onStartCommand(intent: Intent?, flags: Int, startId: Int): Int

Purpose:

- Receive trigger intent
- Read demo mode
- Start payload
- Run foreground service

6.5.2 Mode Detection

The service reads the mode passed from MainActivity.

val mode = intent?.getStringExtra("demo_mode") ?: "notification"

Two execution modes:

Mode	Behavior
notification	show notification
cpu	run CPU workload

Table 6.5.2.1 Execution Modes

This allows conditional payload execution.

6.5.3 Foreground Service Execution

The service runs in foreground mode:

startForeground(1, notification)

Purpose:

- keep service alive
- prevent termination
- run long tasks
- background execution

This allows the service to remain active during the controlled foreground execution window.

6.5.4 Background Thread Execution

The CPU payload runs inside a background thread.

```
Thread {
    // CPU workload
}.start()
```

Purpose:

- avoid blocking UI

- run silently
- simulate malicious task
- background processing

6.5.5 CPU Workload Execution

The service runs a loop for CPU usage simulation.

while (System.currentTimeMillis() < endTime)

This creates:

- CPU spike
- battery drain
- background processing
- attack simulation

6.5.6 Service Stop

After execution completes, the service stops itself.

stopSelf()

This ends the service lifecycle.

7. Security Analysis

This chapter explains the risks of the implemented attack and how developers can prevent reverse engineering and malware injection.

7.1 How Could This Attack Be Prevented?

The Trojan injection attack performed in this project was possible due to lack of application protection mechanisms. Several security techniques can prevent such modifications.

7.1.1 Code Obfuscation

The application source code was easily readable after decompilation. Code obfuscation makes reverse engineering difficult.

Protection methods:

- Rename classes
- Rename methods
- Remove readable strings
- Encrypt logic

Ex:

Original: *DemoService.startCpu()*

Obfuscated: *a.b.c()*

This prevents attackers from understanding code

7.1.2 APK Integrity Checking

The application did not verify its integrity at runtime. Attackers modified and rebuilt the APK without detection.

Protection:

- Verify signature
- checksum validation
- hash comparison

Ex:

App Start
↓
Check APK hash
↓
Modified?
YES
↓
Exit App

These blocks modified APK execution.

7.1.3 Disable Exported Components

Launcher activities may need to remain exported so the Android launcher can start them. However, sensitive logic should not be directly exposed through exported activities. Developers should validate incoming intents, restrict custom intent filters, and avoid placing privileged service-start logic behind externally controllable inputs.

android:exported="true"

Secure configuration:

android:exported="false"

This prevents external access.

7.1.4 Restrict Background Services

The application allowed new services to be added easily. Developers should restrict service usage.

Secure practices:

- permission protected services
- internal services only
- signature level permissions

Ex:

android:permission="com.app.INTERNAL_SERVICE"

This blocks malicious service injection.

7.1.5 Anti Reverse Engineering Protection

The application had no reverse engineering protection.

Protection techniques:

- root detection
- emulator detection
- debugger detection
- tamper detection

Ex:

```
if(isDebuggerAttached())
   exit();
```

This prevents analysis.

7.2 How Can Developers Protect Apps from Reverse Engineering?

Reverse engineering allows attackers to decompile APK files, read source code, and inject malicious components. The Amaze File Manager application was vulnerable because its code and manifest were easily accessible after decompilation. Developers can prevent reverse engineering using the following techniques.

7.2.1 Code Obfuscation (ProGuard / R8)

The decompiled APK contained readable class names such as:

MainActivity
DemoService
CopyService
ExtractService

Obfuscation converts them into unreadable names.

Ex:

Original: *com.amaze.filemanager.ui.activities.MainActivity*
Obfuscated: *a.a.a.a*

This makes reverse engineering difficult.

7.2.2 String Encryption

Sensitive strings were visible in the decompiled APK.

Ex:

- demo_mode
- notification
- cpu
- startForegroundService

Attackers can understand payload behavior.

Secure method:

- encrypt strings
- decrypt at runtime

Ex:
Encrypted: *"ZGVtb19tb2Rl"*
Runtime: *decrypt() → "demo_mode"*

7.2.3 Root Detection
Attackers commonly analyze apps on rooted devices or emulators.
Protection:

```
if (isDeviceRooted())
   exit();
```

This prevents reverse engineering.

7.2.4 Debugger Detection
The app did not detect debugging.
Secure implementation:

```
if (Debug.isDebuggerConnected()) {
   finish();
}
```

This stops dynamic analysis.

7.2.5 Tamper Detection
The APK was modified and rebuilt successfully. Developers should verify signatures.
Flow:

App Start
↓
Check Signature
↓
Modified?
YES
↓
Terminate App

This prevents Trojan injection.

7.2.6 Native Code Protection
Critical logic should be moved to native code (NDK).
Instead of: *Java logic*
Use: *C / C++ native library*

Native code is harder to reverse engineer.

7.3 Mitigation Strategies

This section describes mitigation strategies developers can implement to prevent malware injection, unauthorized service execution, and reverse engineering attacks demonstrated in this project.

7.3.1 Secure Manifest Configuration

The application manifest allowed background services and exported activities that increased attack surface. Developers should restrict component exposure.

Insecure configuration:

android:exported="true"

Secure configuration:

android:exported="false"

Mitigation:

- Disable unnecessary exported components
- Restrict internal services
- Reduce attack surface

7.3.2 Service Permission Protection

The injected DemoService ran without permission restriction. Developers should protect services using custom permissions.

Secure example:

```
<service
   android:name=".SecureService"
   android:permission="com.app.INTERNAL_PERMISSION"
   android:exported="false"/>
```

This prevents unauthorized service execution.

7.3.3 Runtime Integrity Verification

The modified APK ran successfully because the app did not check integrity. Mitigation steps:

App start

↓

Verify APK signature

↓

Modified ?
YES → Exit
NO → Continue

This blocks Trojan injected APKs.

7.3.4 Restrict Foreground Service Usage

The Trojan used foreground service to stay alive.

Mitigation:

- restrict foreground services
- validate service caller
- monitor background execution

Ex:

```
if(!isTrustedCaller())
    stopSelf();
```

This prevents malicious service execution.

7.3.5 Secure Coding Practices

Developers should follow secure coding guidelines.
Recommended practices:

1. validate intents
2. avoid hardcoded logic
3. restrict background execution
4. minimize permissions
5. verify runtime environment

7.3.6 Permission Minimization

The application requested multiple sensitive permissions.
Mitigation:

1. request only required permissions
2. remove unused permissions
3. restrict storage access
4. avoid MANAGE_EXTERNAL_STORAGE

Ex:
Remove:

```
MANAGE_EXTERNAL_STORAGE
QUERY_ALL_PACKAGES
```

This reduces exploitation risk.

8. Testing and Execution Evidence

The modified version of the Amaze File Manager application was then tested within an Android emulator. The objective of this testing phase was to check that the APK had been able to be recompiled, installed, run without issues, and the DemoService could be executed based on the set conditions. Testing also revealed that the application's functionality before injection had not been compromised.

The most important proof of testing was provided through the use of Android Studio, Android Emulator, and Logcat. The screens for build, APK output, application operation, trigger execution, demonstration via notification, CPU demonstration, and logging have been recorded below.

8.1 Build Success

Having made all the required changes in the source code, the next step involved rebuilding the APK using the Android Studio and the Gradle compiler. This process went well, thus, ensuring that there were no syntax errors in the modified code files. It is important to note that this process indicates that the service injection, the manifest, and the triggers were added correctly.

Following this process, an altered F-Droid debug APK file was produced. This APK file was then re-branded as AmazeFileManager_Modified.apk.

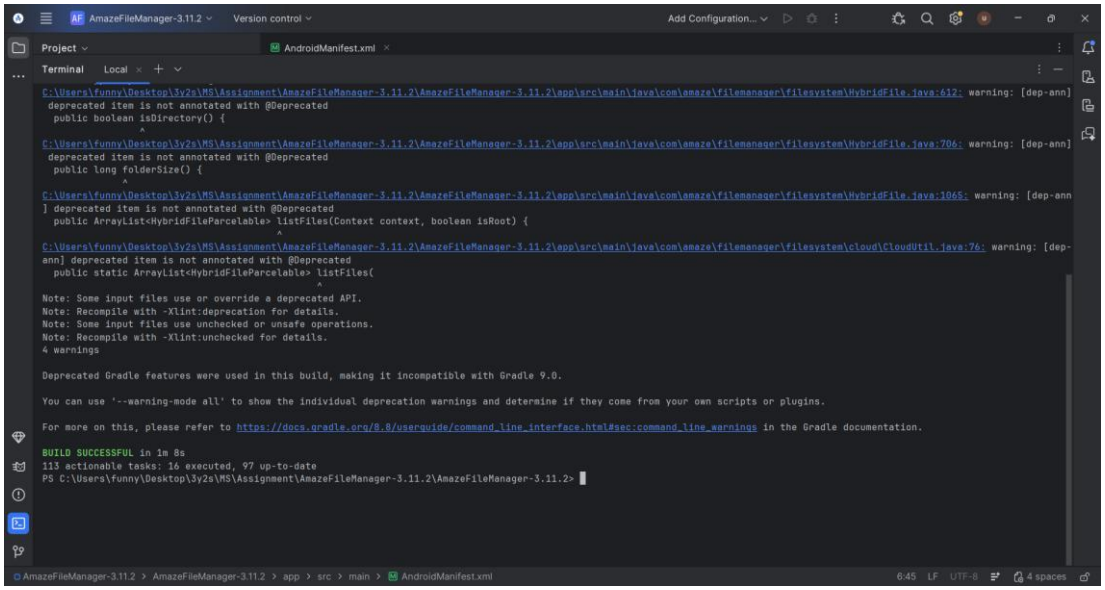

Figure 8.1.1 Successful Gradle build of the modified Amaze File Manager APK

8.2 APK Output

After the successful build, the generated APK was available inside the Gradle output directory. The APK filename was configured as AmazeFileManager_Modified.apk to clearly identify it as the modified version of the application. This file was used for emulator installation and runtime testing.

This confirms that the project was not only modified at source level but also

successfully packaged into an installable Android APK.

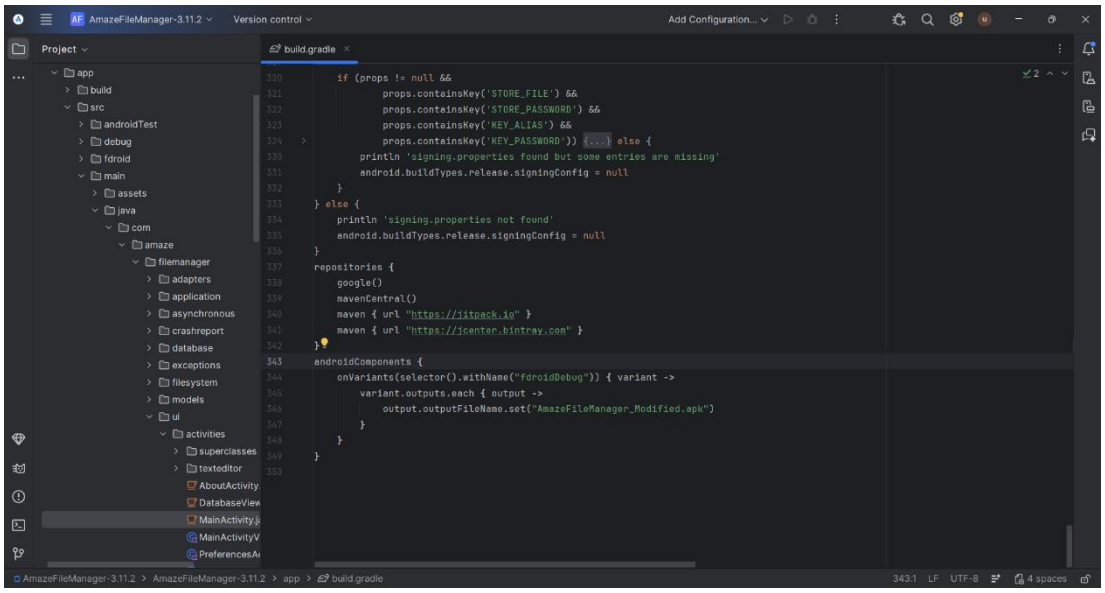

Figure 8.2.1 Modified APK generated as AmazeFileManager_Modified.apk

8.3 App Running Normally

The altered APK package was then installed on the Android Emulator and run. Upon running the application, it showed its usual Amaze File Manager interface where users can browse through their files. The fact that this happened indicates that the application's functionality has been left unharmed by the injected service.

In terms of the interface, the file manager still featured folder options for alarms, Android, audiobooks, DCIM, documents, and download folders. Thus, the application has remained the same even with the new injection.

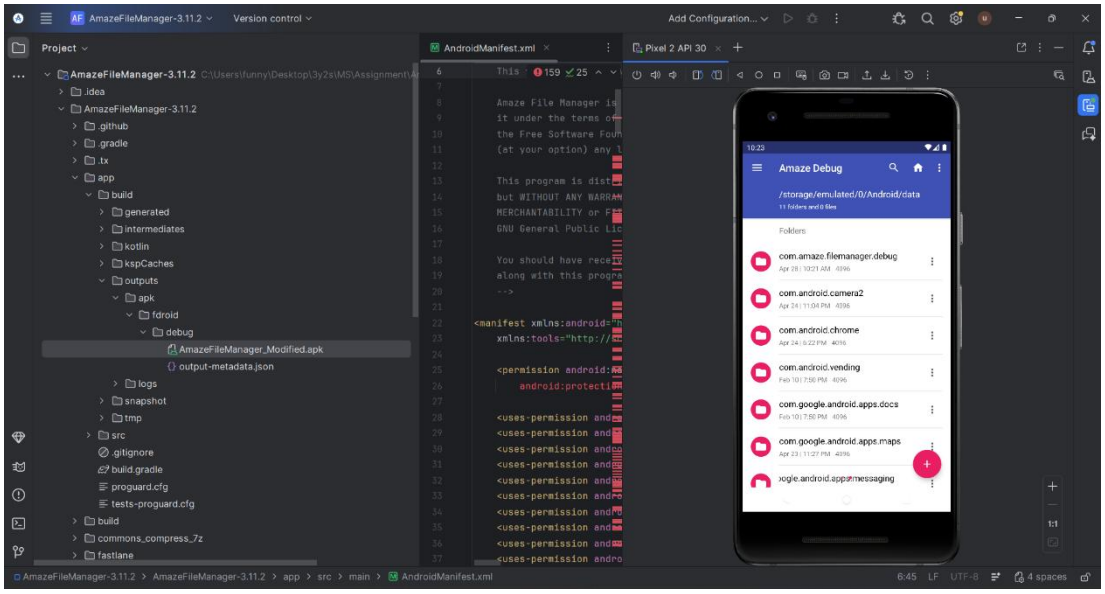

Figure 8.3.1 Modified Amaze File Manager running normally in the emulator

8.4 Notification Demo Execution

Trigger Number 1 was performed via the action of opening the application menu and selecting Run Notification Demo, which triggers the execution of DemoService in the notification mode by specifying demo_mode = "notification" in MainActivity.

This triggered process resulted in the appearance of the foreground service notification in the notification bar of Android. It indicates the successful execution of the injection of the malicious service and the execution of the payload related to the notification mode.

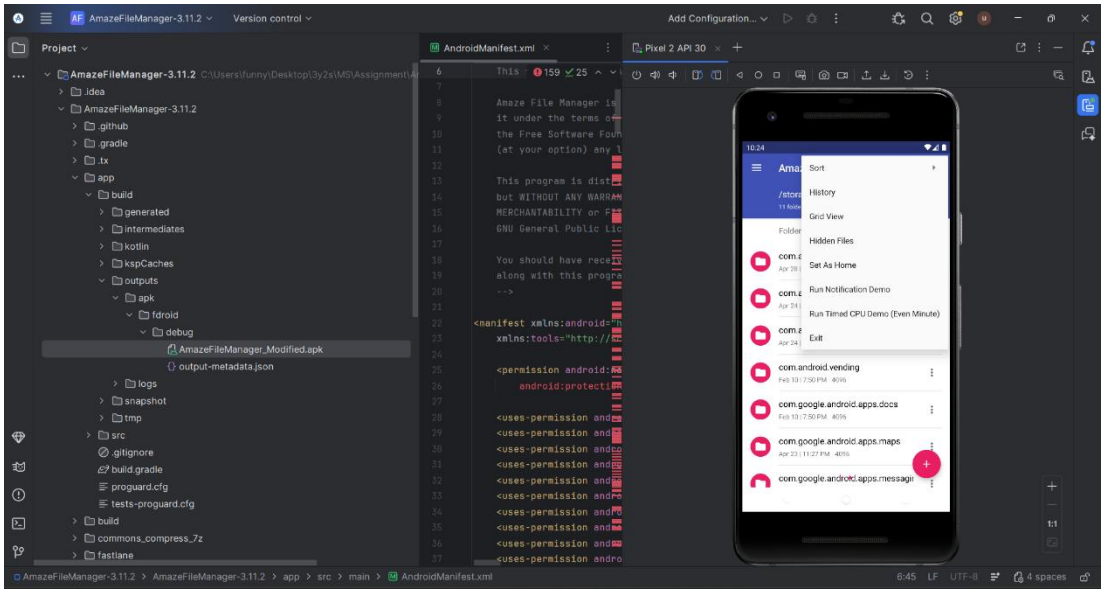

Figure 8.4.1 Added menu trigger options visible in the modified application

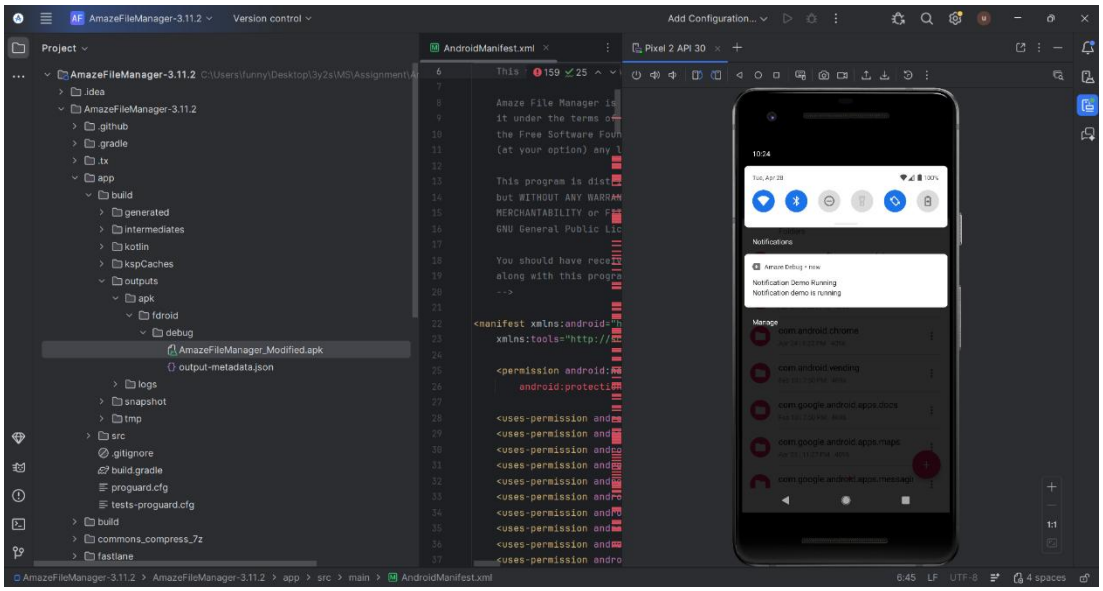

Figure 8.4.2 Notification demo running as a foreground service

8.5 Timed CPU Demo Execution

The second trigger was implemented using the selection of the menu item 'Run Timed CPU Demo (Even Minute)' from the application menu. The second trigger takes into account the present minute at which the service starts. If the present minute is an even one, the demo service will be started, demo_mode set to 'cpu'. Otherwise, If the current minute is odd, DemoService is not started and a toast message is displayed to show that the timed condition was not met.

As can be seen, the second trigger is not simply a copy of the first trigger. Rather, it has conditional activation depending on time. When the second trigger matched, the demo service ran for a limited amount of time, after which it stopped automatically.

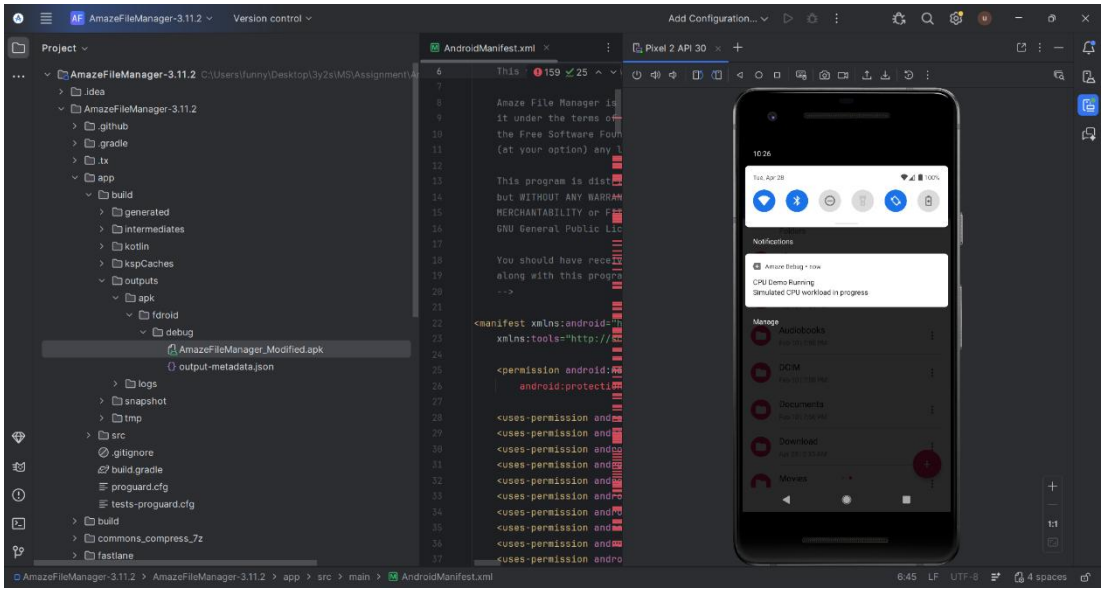

Figure 8.5.1 CPU demo running as a foreground service

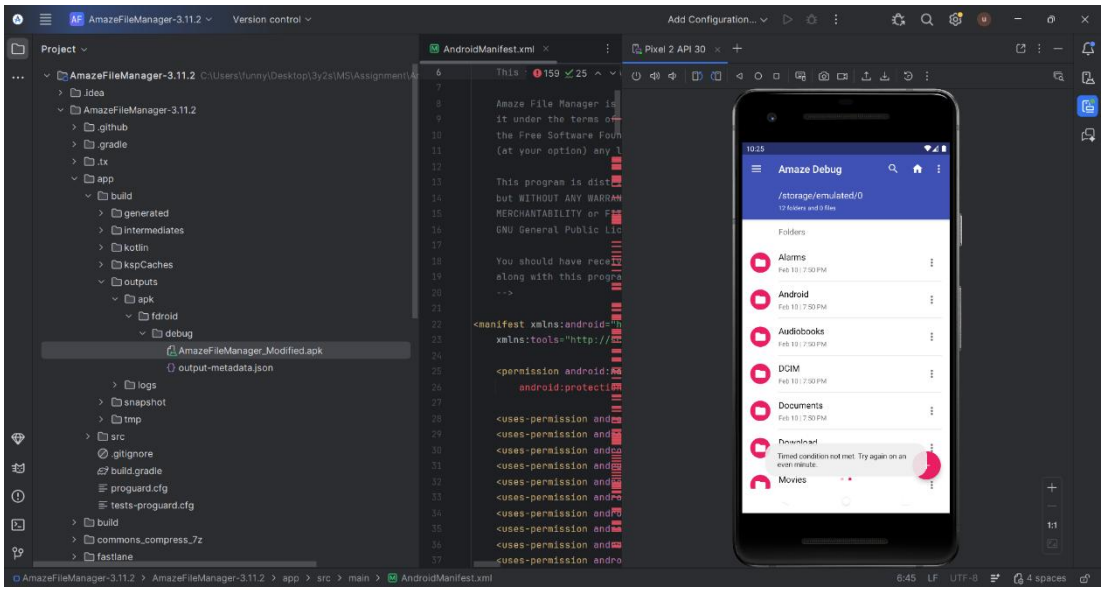

Figure 8.5.2 Timed CPU trigger blocked when the even-minute condition was not met

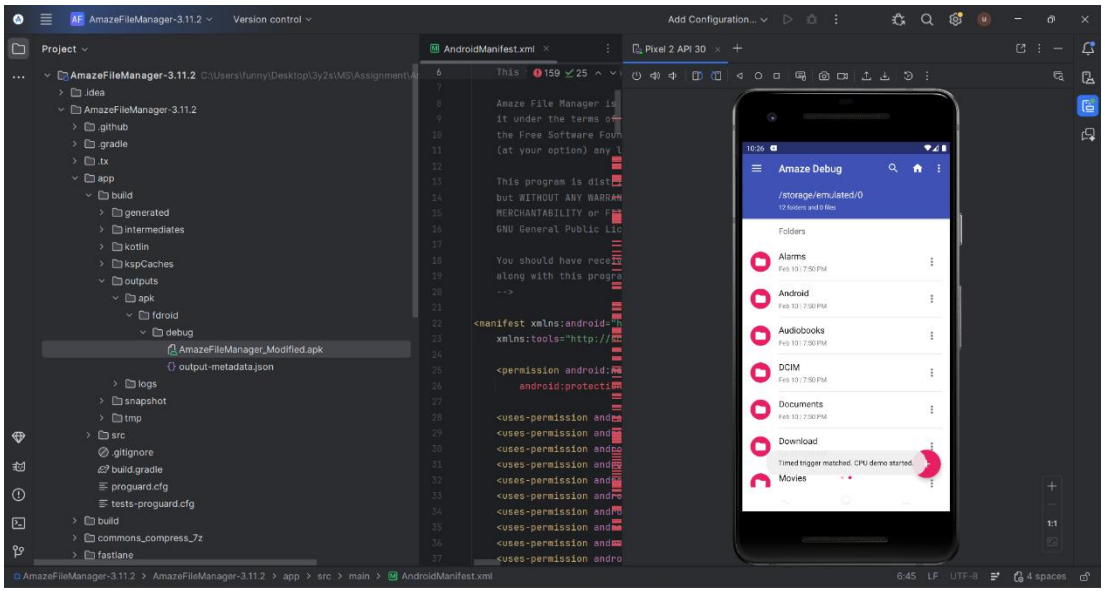

Figure 8.5.3 Timed CPU demo trigger activated on an even minute

8.6 Logcat Verification

Logcat was employed to prove the proper running of the injected service. The logs proved that the DemoService was running, recognized the mode of operation, collected device data, performed the specified action, and stopped its activity.

In case of notification mode, Logcat proved that the service was started with Mode = notification, performed the notification demo, and completed its operation. In the case of the CPU mode, Logcat proved that the service was started with Mode = cpu, performed the CPU task, logged the number of iterations of the loop, and completed its work after about 10 seconds.

CPU Demo log messages are of special importance as they confirm that the CPU task was bounded and finite.

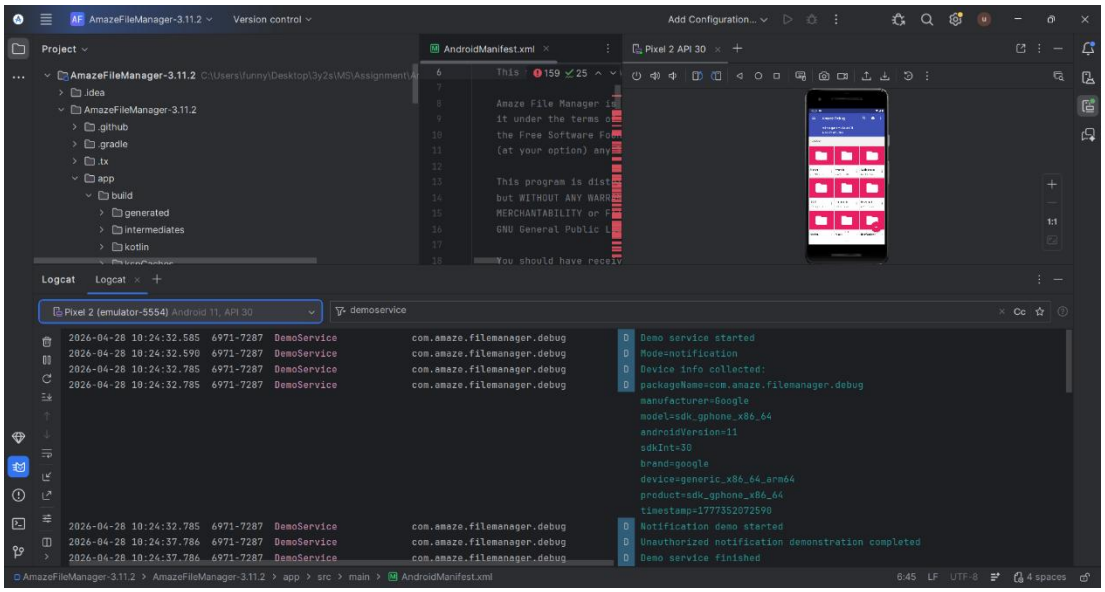

Figure 8.6.1 Logcat output confirming notification mode execution

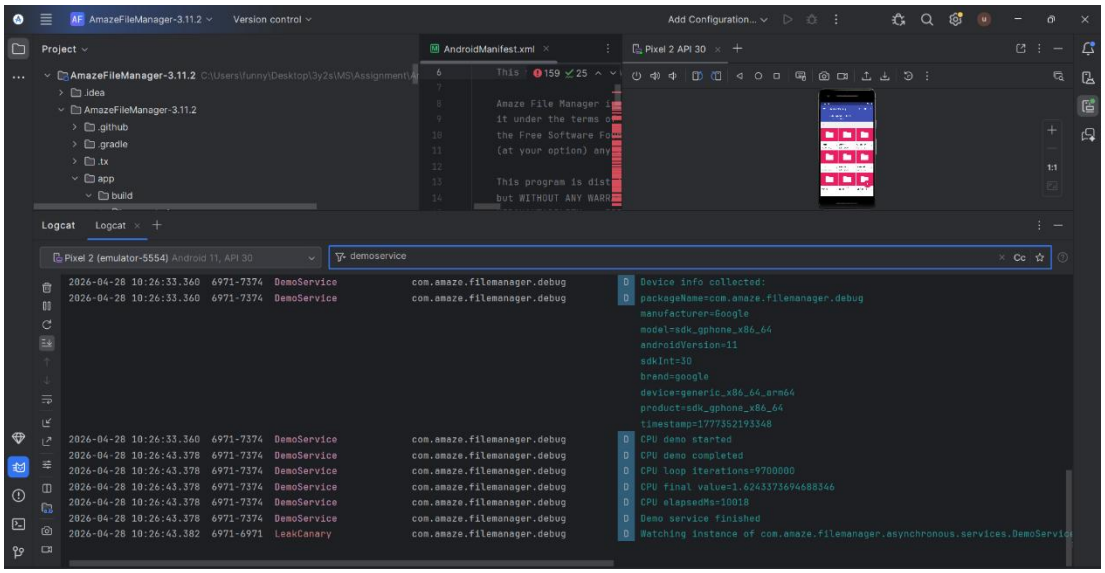

Figure 8.6.2 Logcat output confirming CPU mode completion after approximately 10 seconds

The Logcat log indicates that both modes of payload were executed successfully. Notification mode was able to generate the notification for the foreground service, whereas CPU mode was able to execute controlled computing tasks. As both service execution as well as termination have been observed through the logs, it is clear that both were executed safely.

9. CONCLUSION

Android app reverse engineering and Trojan injection have been carried out using the Amaze File Manager Application. The APK file was reverse engineered using the jadx tool, enabling analysis of the structure of the program, its AndroidManifest.xml file, activities, services, permission sets, and control flow.

The reverse engineering of the program showed that it had a large number of exported components, background services, and permission sets making it highly vulnerable to attack. A malicious service called DemoService has been injected into the program to demonstrate the Trojan injection technique. This service was added to the AndroidManifest.xml file and invoked from MainActivity based on user input and timing conditions.

The altered application was then reconstructed and ran using an Android emulator. The trigger condition effectively launched the malicious service, as proven by the output from Logcat which verified the execution of the injected payload. This illustrates how cybercriminals can tamper with APK files and implant their malicious functions within legitimate applications.

The security assessment revealed that inadequate code obfuscation, lack of file integrity validation, the presence of exported modules, and extensive permissions allowed the exploitation to be successful. There were a number of measures discussed which could have prevented the exploit; among them were code obfuscation, tampering prevention, minimized permissions, protection of services, and safe manifest configuration.

In conclusion, it should be clear that this case study underscores the significance of developing secure Android applications and ensuring the use of protective mechanisms against any potential security threats.

APPENDIX A: INJECTED MALICIOUS SERVICE (DemoService.kt)

This appendix contains the complete source code of the injected Trojan service used to simulate malicious background execution. The service collects device information, executes a CPU-intensive loop, and displays a foreground notification.

This component was added to the following directory:

com/amaze/filemanager/asynchronous/services/DemoService.kt

The service is triggered from MainActivity and runs as a foreground service. Two execution modes are implemented:

- Notification mode
- CPU workload mode

The service lifecycle includes:

- onStartCommand() trigger
- mode detection
- foreground execution
- background thread
- CPU workload
- stopSelf() termination

This appendix provides the full implementation used during malware injection.

APPENDIX B: MODIFIED MAINACTIVITY TRIGGER LOGIC

This appendix includes the modified MainActivity.java code used to trigger the malicious service. The trigger was implemented using menu options and a time-based conditional check.

Two triggers were added:

1. Notification trigger
2. Even-minute CPU trigger

Trigger logic:

- User selects menu item
- MainActivity creates intent
- demo_mode parameter passed
- DemoService started
- Payload executed

Time-based condition:

- System minute retrieved
- Even minute check applied
- CPU payload executed only if condition satisfied

This appendix demonstrates how the malicious component is activated from the UI.
Ex:

```
public boolean onCreateOptionsMenu(Menu menu) {
  MenuInflater menuInflater = getMenuInflater();
  menuInflater.inflate(R.menu.activity_extra, menu);

    menu.add(Menu.NONE, R.id.menu_demo_service, Menu.NONE, "Run
Notification Demo");
    menu.add(Menu.NONE, 9002, Menu.NONE, "Run Timed CPU Demo (Even
Minute)");
    return true;
}
```

Ex:

```
public boolean onOptionsItemSelected(MenuItem item) {
 if (item.getItemId() == R.id.menu_demo_service) {
     Intent demoIntent =
            new Intent(this,
```

```
com.amaze.filemanager.asynchronous.services.DemoService.class);
    demoIntent.putExtra("demo_mode", "notification");
    ContextCompat.startForegroundService(this, demoIntent);
    Toast.makeText(this, "Notification demo started.",
Toast.LENGTH_SHORT).show();
    return true;
 }
 if (item.getItemId() == 9002) {
    Calendar calendar = Calendar.getInstance();
    int minute = calendar.get(Calendar.MINUTE);

    if (minute % 2 == 0) {
        Intent demoIntent =
                new Intent(this,
com.amaze.filemanager.asynchronous.services.DemoService.class);
        demoIntent.putExtra("demo_mode", "cpu");
        ContextCompat.startForegroundService(this, demoIntent);
        Toast.makeText(this, "Timed trigger matched. CPU demo started.",
Toast.LENGTH_SHORT).show();
    } else {
        Toast.makeText(this, "Timed condition not met. Try again on an even minute.",
Toast.LENGTH_SHORT).show();
    }
    return true;
 }
```

Ex:

```
import java.util.Calendar;

if (item.getItemId() == 9002) {
   Calendar calendar = Calendar.getInstance();
   int minute = calendar.get(Calendar.MINUTE);

   if (minute % 2 == 0) {
      Intent demoIntent =
              new Intent(this,
com.amaze.filemanager.asynchronous.services.DemoService.class);
      demoIntent.putExtra("demo_mode", "cpu");
      ContextCompat.startForegroundService(this, demoIntent);
      Toast.makeText(this, "Timed trigger matched. CPU demo started.",
Toast.LENGTH_SHORT).show();
   } else {
```

```
        Toast.makeText(this, "Timed condition not met. Try again on an even minute.",
Toast.LENGTH_SHORT).show();
    }
    return true;
}
```

APPENDIX C: MODIFIED AndroidManifest.xml

This appendix contains the AndroidManifest.xml modifications used to register the injected malicious service and required permissions.
The following changes were made:

Service Registration:

- DemoService added
- Foreground service enabled
- dataSync service type

Permissions Used:

- FOREGROUND_SERVICE
- FOREGROUND_SERVICE_DATA_SYNC
- POST_NOTIFICATIONS
- WAKE_LOCK

These modifications allow the Trojan service to run persistently in the background.

Ex: *<service*
android:name=".asynchronous.services.DemoService"
android:foregroundServiceType="dataSync"
android:exported="false"/>

This appendix shows how the malicious component is integrated into the application lifecycle.